

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
University M'Hamed BOUGARA – Boumerdes



Institute of Electrical and Electronic Engineering
Department of Electronics

Project Report Presented in Partial Fulfillment of
The Requirement for the Degree of

Master

In Computer Engineering

Title:

**Transformer-Based Approach for Hand Gesture
Recognition Through EMG Signals**

Presented by:

Rayane MESSALIT

Yacine SIF

Supervisor:

Dr. Elhocine BOUTELLAA

Co-supervisor:

Dr. Rachid NAMANE

Academic Year: **2024/2025**

Acknowledgments

In the name of Allah, the Most Beneficent and the Most Merciful, we begin by offering our sincere gratitude to Allah for the countless blessings and the strength he granted us to bring this work to fruition.

Our deepest appreciation goes to our supervisor, **Dr. Elhocine Boutellaa**, whose expert guidance, unwavering support, and insightful feedback were instrumental throughout every stage of this research. We also thank our co-supervisor, **Dr. Rachid Namane**, for his occasional advice, encouragement, and thoughtful insights, which contributed to the quality of this work.

We take pride in acknowledging our own perseverance, enthusiasm, and tireless effort in conducting the research and assembling this project, despite the numerous challenges encountered over the year.

We extend our heartfelt gratitude to our families and friends for their faith in us and their constant encouragement. Their patience and understanding provided the foundation we needed to stay focused and motivated.

Finally, we express our thanks to all our colleagues who directly or indirectly contributed to the successful completion of this work, their support and expertise were vital at every step.

Rayane MESSALIT, Yacine SIF

Abstract

Hand gesture recognition using Electromyography (EMG) signals is a crucial area of research for advancing human-computer interaction, prosthetic control, and assistive technologies. Traditional machine learning and deep learning models have demonstrated success, but they struggle in capturing long-range dependencies within the time-series nature of EMG signals. This project introduces a Transformer-based architecture for the classification of hand gestures from EMG signals, leveraging the self-attention mechanisms to effectively model temporal relationships in EMG data, potentially improving recognition accuracy and robustness. This study utilized the Ninapro DB2 dataset, a publicly available dataset. We address the challenges of EMG signal variability, noise, and inter-subject differences through a robust pre-processing pipeline and a custom model design. Our approach incorporates a convolutional feature extractor, Time2Vec positional encoding, and a Transformer encoder to capture complex temporal dependencies. The model was implemented and trained to recognize a set of 15 distinct hand gestures. Experimental results demonstrate an average test accuracy of 82.98% and a Matthews Correlation Coefficient (MCC) of 0.8178, consistently outperforming conventional models included in the comparison. Additionally, our model demonstrates exceptional parameter efficiency, achieving superior performance with moderate complexity (128,536 parameters) compared to other over-parameterized models evaluated, highlighting the effectiveness of our Transformer-based architecture in balancing performance and parameter efficiency. The model computational efficiency, with an average testing time of 1.86 ms per window, highlights its suitability for real-time applications in human-machine interfaces. This research highlights the promise of Transformers in learning discriminative features from complex biosignals, paving the way for more intuitive and reliable control of external devices.

Keywords: Hand gesture recognition, Electromyography (EMG), Deep learning, Transformer model, Time series classification.

Contents

Acknowledgments	i
Abstract	ii
List of Abbreviations	viii
Introduction	1
1 Upper Limb Anatomy and EMG Signals	3
1.1 Introduction	3
1.2 Upper Limb Muscles	3
1.2.1 Structure and Function	3
1.2.2 Nerve Control and Muscle Activation	3
1.3 Electromyography	4
1.3.1 Motor Unit Action Potentials	5
1.3.2 Electromyographic Signals Acquisition	5
1.3.3 Electromyographic Signals Acquisition Challenges	8
1.3.4 Hand Gesture EMG Datasets	9
1.4 Conclusion	10
2 Deep Learning for EMG-based Hand Gesture Recognition Systems	11
2.1 Introduction	11
2.2 The Transformer Architecture	11
2.2.1 Encoder-Decoder Structure	11
2.2.2 Multi-Head Attention	12
2.2.3 Transformer for Time Series	14
2.2.4 The Quadratic Complexity Issue	14
2.3 Related Works	15
2.4 Conclusion	19
3 Data Preparation and Proposed Model Architecture	20
3.1 Introduction	20
3.2 NinaPro Dataset	20
3.2.1 Important Considerations for the NinaPro Dataset	22
3.2.2 Gestures Selection	23
3.3 Data Pre-processing	25
3.4 Data visualization	27
3.5 Proposed Model Architecture	30

	3.5.1 Overall Network Diagram Summary	30
	3.5.2 Convolutional Feature Extractor	31
	3.5.3 Time2Vec Positional Encoding	32
	3.5.4 Transformer Encoder	32
	3.5.5 Classification Head	33
	3.6 Conclusion	34
4	Proposed Model Evaluation and Results	35
	4.1 Introduction	35
	4.2 Implementation Details	35
	4.2.1 Libraries	35
	4.3 Test Environment	36
	4.4 Evaluation Metrics	37
	4.5 Experimental Results and Discussion	38
	4.5.1 Hyperparameter Tuning Experiments	39
	4.5.2 Effect of the Pre-processing Techniques on the Model Performance	52
	4.5.3 Time Complexity Analysis	53
	4.5.4 Performance Comparison with Other Models	54
	4.6 Conclusion	57
	Conclusion	58
A	Main Deep Learning Architectures	59
	A.1 CNN-based Methods	59
	A.2 RNN-based Methods	62
	A.2.1 CNNs and RNNs in Time Series Classification	64
	A.2.2 CNNs and RNNs Limitations for Time Series Classification	64
	A.3 Training Parameters	65
B		66
	B.1 Technologies	66

List of Figures

1.1	Example of the Surface EMG Signal Measurement [17].	5
1.2	Ottobock MyoBock Electrodes [20].	7
1.3	Trigno Avanti Sensor [21].	7
1.4	Inserting and Removal of the Fine-Wire [22].	8
2.1	The Transformer Model Architecture [30]	12
2.2	Multi-Head Attention, Consisting of Several Attention Layers Running in Parallel [30]	13
3.1	End-to-end System Overview.	20
3.2	Frequency Response of Band Pass Filter [43].	26
3.3	Notch Filter Characteristics [46].	27
3.4	Effect of the Pre-processing Techniques on the EMG Signal in Time Domain.	28
3.5	Effect of the Pre-processing Techniques on the EMG Signal in Frequency Domain.	29
3.6	Effect of the 50 HZ Notch Filter on the EMG Signal.	30
3.7	Overview of the Proposed Transformer Model.	31
3.8	Convolutional Feature Extractor Layers.	32
3.9	Attention and Normalization Block Layers.	33
3.10	Feed Forward Block Layers	33
3.11	Classification Head	33
4.1	Accuracy and Loss Performance Over Epochs	40
4.2	Model Test Accuracy Improvement Through Optimization	46
4.3	Global Confusion Matrix of Final Model	47
4.4	Train vs Test Accuracy (Color = MCC)	49
4.5	Confusion Matrices and Training Curves for Best (S33) and Worst (S24) Subjects Using the Final Model Configuration.	51
4.6	Comparison of the Model Performance on Raw and Preprocessed Data	52
4.7	Real-Time Feasibility Timeline	54
4.8	Highest and Lowest Test Accuracies Across Models with Corresponding Subjects.	56
4.9	Test Accuracy Across All Subjects: Proposed Model vs. CNN-1D. . .	56

A.1	Single Channel Convolution Visualization [50].	60
A.2	Multi-channel Convolution Visualization[50]	61
A.3	Example of Pooling Layer Visualization [50].	61
A.4	With and Without Dropout[52]	62
A.5	Comparison of Feedforward and Recurrent Neural Networks [54]. . . .	63
A.6	RNN Unfolded[54]	63

List of Tables

2.1	Summary of the EMG Hand-gestures Classification Studies.	18
3.1	Exercises with number of gestures available [41].	21
3.2	Comparison of NinaPro DB1 to DB5 Datasets	22
3.3	List of Selected Hand Movements With Corresponding Gesture Images.	24
3.4	SNR Improvement Throughout EMG Signal Processing Pipeline.	30
4.1	Confusion Matrix of a Classification Problem.	37
4.2	Initial Model Key Hyperparameters Configuration	39
4.3	Number of Epochs Tuning Experiments Results	39
4.4	Learning Rate and Batch Size Tuning Experiments Results	41
4.5	No. of Heads and Head Size Tuning Experiments Result	42
4.6	MLP and FF Dimension Tuning Experiments Result	43
4.7	Encoder Blocks Tuning Experiments Result	44
4.8	Dropout and Bias Regularization Tuning Experiments Result	45
4.9	Performance Metrics for Subjects 1–40	48
4.10	Results for Best (S33) and Worst (S24) Subjects for the Final Model Configuration	50
4.11	Model Results on Raw Data	52
4.12	Training and Testing Time Metrics Across 40 Subjects	53
4.13	Performance and Complexity Comparison of the Models	55

List of Abbreviations

Ag-AgCl	Silver-Silver Chloride
BPF	Bandpass Filter
CNN	Convolutional Neural Network
Conv1D	1D Convolutional Layer
CUDA	Compute Unified Device Architecture
DB	Database
DCGAN	Deep Convolutional Generative Adversarial Network
EMG	Electromyography
FF	Feed-Forward
FN	False Negative
FP	False Positive
HMI s	Human-Machine Interfaces
HPF	High Pass Filter
LSTM	Long Short-Term Memory
LPF	Low Pass Filter
MCC	Matthews Correlation Coefficient
MLP	Multilayer Perceptron
MUAP s	Motor Unit Action Potentials
NLP	Natural Language Processing
PLI	Power-Line Interference
ReLU	Rectified Linear Unit

RNN	Recurrent Neural Network
SNR	Signal-to-Noise Ratio
TN	True Negative
TP	True Positive

Introduction

Electromyographic (EMG) signals, which are the electrical potentials generated during muscle activation, are foundation for advanced human-machine interfaces (HMIs) [1]. By capturing the electrical activity of muscles, EMG based systems enable intuitive control over devices without reliance on cameras or handheld controllers in a wide range of applications, ranging from medical applications such as prosthetic control to virtual realities [1]. Despite their promise, utilizing trustworthy EMG systems remains a topic of the future: recordings are inherently variable as a result of movement noise from the electrodes, muscle fatigue, motion artifact, as well as physiological variation among users [2, 3]. These sources affect model performance adversely, particularly when generalizing to new subjects [4].

Traditional deep learning approaches have been established as practical choices for EMG classification by leveraging their ability to extract local spatial and temporal features [5], but their potential for capturing long-term dependencies in noisy time series data remains restricted [6]. To tackle such issues, Transformers offer a compelling alternative. Their self-attention mechanisms dynamically weigh important signal regions and model contextual associations over long sequences [7]. Although originally designed for natural language processing (NLP), transformers are now increasingly used in physiological time series processing [8] and offer a potentially attractive framework to handle the variabilities of the EMG while maintaining real-time capabilities in embedded hardware [9].

This Master project main focus involves exploring the transformer architecture with its unique self attention mechanisms and ability to model long range dependencies by observing its ability to tackle the challenges in EMG based hand gesture recognition. This work aims to contribute to the broader field of intuitive human machine interfaces by significantly help in advancing applications that can extend from medical fields such as prosthetic control to immersive virtual and augmented reality systems and everyday consumer technology.

Problem Statement

Electromyographic (EMG) based hand gesture recognition has the potential to revolutionize multiple domains, from medical prosthetic control to gaming interfaces and general human computer interaction. Yet prevailing deep learning approaches struggle with long range dependencies arising from muscle fatigue, motion artifacts, and subject differences [5, 6]. This issue imply a considerable concern in the medical applications, where reliability is an overriding priority. Amputees, as an example, exhibit very idiosyncratic residual activation patterns and unique electrode placement constraints, leading to very poor performance degradation when a model learned under one subject is run under another [4]. Similarly, in general consumer

applications, the physiological characteristics differences across the consumers create obstacles in developing universally responsive interfaces that work reliably without extensive personalization [10]. Failure to extract such relationships undermines their ability to generalize from one subject to another.

While transformer architectures, address these issues through their global self-attention [7], its extension to EMG time-series classification continues to be a new frontier. There are fundamental questions as to if it can deal with the high noise, high variability, and timeliness demands of EMG systems, in numerous domains in which stability and cross-subject generalizability are of overriding importance. Closing that knowledge gap is essential to enabling next generation human computer interfaces that are both reliable and accessible to many people.

Objectives

In this project, a transformer-based architecture specially tailored to the unique challenges of EMG time-series classification will be designed and validated. The model's capabilities in handling critical aspects such as classification accuracy and cross-subject generalizability will be demonstrated to observe the model effectiveness in processing EMG time-series data. Through experiments on EMG dataset, the capabilities of the transformer to improve EMG-based systems for practical applications in medical field or in any domain requiring precise gesture recognition, real-time responsiveness, and cross-user adaptability, to immersive human machine interaction shall be established.

Chapter 1 Upper Limb Anatomy and EMG Signals

1.1 Introduction

The chapter presents the anatomy and physiology of the upper limb muscles, including their structure, function, and the sophisticated mechanisms that allow movement. It begins by investigating how these muscles are grouped into anatomical compartments and how sarcomeres and myofibrils produce force. Next, it explores how the nervous system controls these muscles, including motor neurons and motor units responsible for activating the muscle. Finally, the concept of electromyography (EMG) signals will be introduced, including EMG instrumentation and the challenges of signal acquisition. This will be followed by a review of the most widely used EMG datasets for hand gesture recognition, highlighting their unique characteristics and differences.

1.2 Upper Limb Muscles

1.2.1 Structure and Function

Numerous muscles that are arranged into anatomical compartments make up the upper limb. These muscles maintain tone, offer stability, and enable precise fluid movement in the hand, arm, and shoulder joints [11]. Muscles in the upper limbs and shoulders are specialized for applying pressure and manipulating objects. Skeletal muscles contain bundles of myofibrils made up of sarcomeres, which are the fundamental contractile units that generate force. Additionally, skeletal muscle performs a variety of tasks, including voluntary movement, internal organ protection, heat production, and support for posture [11].

Muscle mass depends on the equilibrium between protein production and breakdown, where the control of this process is sensitive to several factors such as nutritional status, hormone levels, physical activity, underlying diseases, and injuries [12].

1.2.2 Nerve Control and Muscle Activation

Voluntary movement begins with a motor command from the cortex, which travels down the spinal cord, exits via an alpha motor neuron, and triggers muscle contraction. The central nervous system controls every skeletal muscle with an alpha

motor neuron. An alpha motor neuron cell is located in the spinal cord in the ventral horn [13]. The axon exits through the ventral root and inserts into muscle fiber. The motor neuron axon branches many times after entering the target muscle; each branch forms a small cluster of terminals that contact different muscle fibers. When a motor neuron fires, all the muscle fibers in that motor unit contract simultaneously. The size of a motor unit varies from just a few fibers in the precise movement of the eye muscles to more than a thousand fibers in the large movements of the leg muscles [13].

The understanding of neural activation and mechanical properties provides the basis for interpreting electromyographic (EMG) signals, which reflect muscle activity. EMG measures the electrical activity generated by skeletal muscles during contraction. These electrical signals originate from the neural commands sent by alpha motor neurons.

1.3 Electromyography

Electromyography (EMG) is a diagnostic technique used to record bioelectric activity produced by muscle contraction in skeletal muscles. This technique generally includes stimulation of relevant peripheral and motor neurons, with measurements made at various levels ranging from a single muscle fiber or a group of motor units to entire muscles through either invasive or non-invasive methods. The information obtained with EMG is used to diagnose neuromuscular and muscular diseases [14], in planning rehabilitation programs [15], or in controlling prosthetic devices [16].

Electrode configurations used in electromyography (EMG) can be monopolar, bipolar, or even hybrid combining both surface and intramuscular electrodes. Multi-channel systems are used mainly in surface recordings, where electrodes are systematically placed in stripe or matrix arrangements on a silicone sheet. To reduce interference, a grounding electrode is placed far from the recording site, and a right-leg drive configuration (as in ECG) may be employed to reduce interference (see Figure 1.1). The frequency band of EMG signals ranges from 0.01 Hz to 10 kHz, with noninvasive methods working mainly within the 50 to 150 Hz range identified as the most clinically relevant [17].

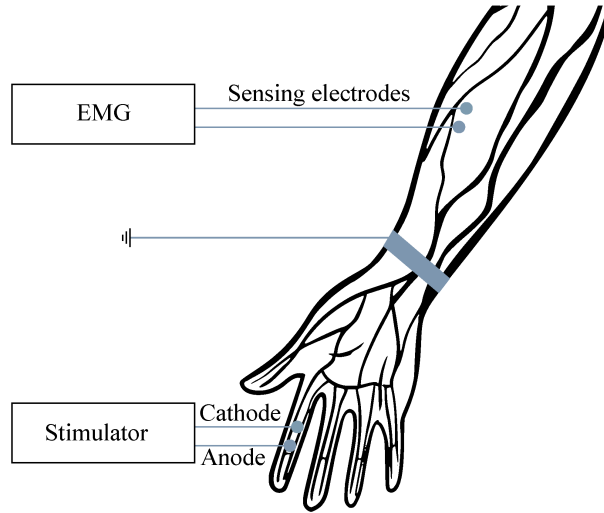


Figure 1.1: Example of the Surface EMG Signal Measurement [17].

1.3.1 Motor Unit Action Potentials

Motor Unit Action Potentials, or MUAPs, are essentially the summation of electrical activity from all the muscle fibers within one motor unit, providing us with some useful information on muscle functioning. Their appearance on a graph is the result of summing individual muscle fiber action potentials, and they are crucial for identifying neuromuscular disorders, as changes in MUAP shape and firing rate can indicate pathology [18]. MUAPs can be recorded invasively by inserting needles or wires directly into the muscle or non-invasively with surface electrodes that record a mixed signal, generally with a bit of additional noise from adjacent motor units. In EMG analysis, MUAPs are often modeled as filtered impulse trains. This framework enables detailed decomposition for clinical diagnostics, prosthetic control, grip recognition, and other human-machine interfacing technologies.

1.3.2 Electromyographic Signals Acquisition

Electromyographic signals, representing the electrical activity generated by muscle tissues, can be acquired using various electrode configurations and techniques. This section introduces the types of electrodes used for EMG signal detection.

EMG Electrodes Types

EMG signals can be detected using various electrode types. In this section, we discuss two main categories: surface electrodes (which are further divided into gelled and dry types) and inserted electrodes (including needle and fine-wire variants)

Surface Electrodes (Gelled vs. Dry): Surface electrodes are a type of electrodes that are placed on the skin of the subject for measurement and detection of EMG signals. Two types of surface electrodes are commonly in use:

- Dry electrodes, which make direct contact with the skin.

- Gelled electrodes, which uses an electrolytic gel as a chemical interface between the skin and the metallic part of the electrode [19].

Gelled Electrodes: Gelled electrodes use an electrolytic gel as the interface between skin and the metallic element. Oxidative and reductive chemical reactions occur at the metal surface-gel interface. Silver-Silver-chloride (Ag/AgCl) is the most common composite for the metallic part of gelled electrodes. The AgCl layer allows current from the muscle to pass more freely across the junction between the electrolyte and the electrode. This introduces less electrical noise into the measurement, as compared with equivalent metallic electrodes (e.g. Ag). As a result, Ag/AgCl electrodes are used in over 80% of surface EMG applications [19].

Gelled electrodes can be disposable or reusable. They are widely used due to their lightweight design. they are available in a variety of shapes and sizes, as well as for materials used for the patch and conductive gel. Proper application of disposable electrodes reduces the possibility of displacement, especially during rapid movements.

Dry Electrodes: Dry electrodes are used directly in contact with the skin and do not require a gel like the gelled electrodes do. They typically require pre-amplifier circuitry at the electrode site due to their high electrode-skin impedance. Dry electrodes (typically $> 20g$) are considerably heavier than gelled electrodes ($< 1g$). This increased inertial mass increases the difficulty in maintaining electrode fixation, as compared to gelled electrodes [19]. Hereafter, are some devices that use dry electrodes:

- Ottobock [20] is a German company specialized in producing myoelectric prostheses. One of its products is the MyoBock electrode. These electrodes are more sensitive to low muscle signals compared to other electrodes. In the range of high muscle signals the differentiation of the signal level is better since the change in amplification now happens logarithmically. Furthermore, due to modern frequency shielding and filtering technologies it is significantly less sensitive to low and high frequency interferences that are emitted, for example, by mobile phones or shopping center security systems.



Figure 1.2: Ottobock MyoBock Electrodes [20].

- Delsys is an American company founded in 1993 and is focused on solving the engineering challenges associated with wearable EMG sensors. These challenges include: low signal artifact, low crosstalk, signal reliability, and signal consistency. Their products serve a critical role in assisting thousands of researchers and educators in 85 countries worldwide to investigate and solve human movement disorder issues. One of its products is Delsys Trigno [21], which is a wireless EMG and motion sensor system. The sensor is ideal for real-time applications since it is wireless and some models include accelerometers, gyroscopes, and magnetometers for motion tracking.

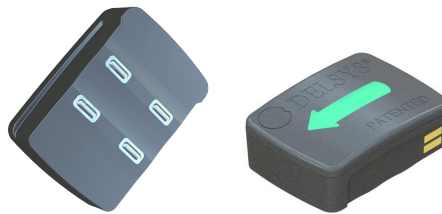


Figure 1.3: Trigno Avanti Sensor [21].

Inserted Electrodes: They are generally made of stainless steel. These electrodes are designed to penetrate the skin surface of the body to some depth to record EMG potentials of a muscle. These electrodes have to be sharp and fine like subdermal needles, allowing them to penetrate the skin and superficial fascia to reach the muscle. Two types of inserted electrode are introduced in the following:

- Fine-wire electrodes, which consist of very thin wires that are inserted into the muscle tissue.
- Needle electrodes, similar to fine-wire electrodes in insertion method but differing in design and clinical application.

Fine-Wire Electrodes: In kinesiological studies, human movement may cause problems. Therefore, thin and flexible fine-wire electrodes are the preferred choice for invasive electrode application within deeper muscle layers. The wires are inserted by hollow needles and their proper localization can be tested by electrical stimulators or ultrasound imaging. The fine-wire is done by inserting a needle carefully into the muscle at the selected site. After removing the needle, the distal endings of the wires are connected to steel spring adapters, which again are connected to the regular EMG pre-amplifier lead [22]. Fine-wires are easy to insert and to remove from the skeletal muscles and they cause less pain compared to the needle electrodes, which stay within the muscle for the entire test [23].

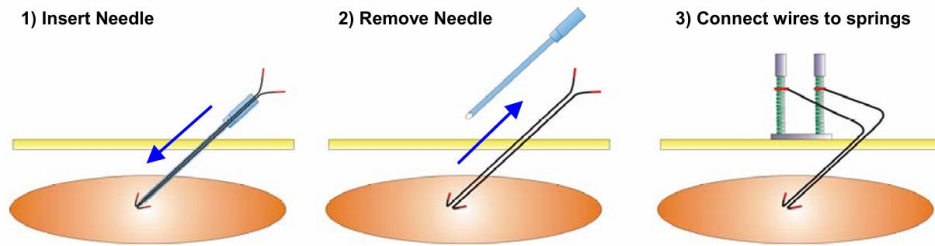


Figure 1.4: Inserting and Removal of the Fine-Wire [22].

Needle Electrodes: can be used to record and amplify the electrical signals generated from resting or contracting muscle fibers after inserting a needle electrode into a muscle and interpreting the signals to determine the function of the muscle fibers and motor units. Needle electromyography is generally a safe procedure. However, since the needle is inserted into the muscle, there are potential risks associated with movements, including pain, bleeding, hematoma formation, infection, and development of pneumothorax [24].

1.3.3 Electromyographic Signals Acquisition Challenges

Electromyography (EMG) signal acquisition is fraught with numerous challenges, each impacting the quality and reliability of the recorded data. Below are some examples of these challenges:

- Cross-talk [25]: the signals captured from unrelated muscles, that are “mixed in” with the electrical signals of the muscle of interest, are referred to as cross-talk. Cross-talk is commonly encountered when using surface electrodes. They can only be used effectively on the superficial and large muscles to provide a stable contact area for the electrodes to mount properly, meaning that cross-talk is more problematic when dealing with smaller muscles within a complex mechanical arrangement, such as the forearm. Using surface EMG detection to isolate the activity of a single muscle is challenging because the entire limb is a conductive volume, electrical activity from distant muscles can be conducted through intervening tissues to the recording electrode. The electrical activity of muscles located anywhere within the limb may be conducted through the

intervening tissue to reach the electrode, even if they are some distance away from the skin surface.

- **Movement Artifact [26]:** Motion or movement artifact in EMG signals can arise from cable movement and the shifting interface between electrodes and the skin. These artifacts can have amplitudes similar to the EMG signal and usually occur in the 1–10 Hz frequency range. Recessed electrodes are a type of surface electrode designed to reduce motion artifacts in EMG recordings. Small groove or indentation, use a special gel to improve contact with the skin. This reduces undesired signals caused by electrode movement or motion artifacts. The EMG signal may still be impacted by interference from electrical differences between different layers of the skin, which these electrodes cannot stop.
- **Inherent Noise in the Electrode [26]:** All electronic devices produce electrical noise, also known as “inherent noise”, which spans a frequency range from 0 Hz to several thousand Hz. For recording purposes, electrodes made of silver/silver chloride (10×1 mm) have been found to give adequate signal-to-noise ratio and are electrically very steady. The impedance decreases with increasing electrode size, however the size shouldn’t be excessively large. On the other hand, high electrode impedance might reduce the signal-to-noise ratio, so the impedance and size must be carefully balanced. Higher impedance is suitable for experiments that need a greater number of electrodes or high statistical power. However, lower impedance is preferable for experiments when signal quality is important. Employing advanced circuit design and high-quality instruments can help minimize this noise.

1.3.4 Hand Gesture EMG Datasets

Electromyography (EMG) datasets are collections of recordings that capture the electrical activity produced by skeletal muscles. Several publicly datasets are available for research and application purposes. We will explore publicly available EMG datasets for hand gesture recognition, by providing a comparative analysis of key datasets.

- **BioPatRec Dataset [27]:** BioPatRec is a publicly available dataset that is associated with the open-source platform for pattern recognition of myoelectric signals BioPatRec project. The dataset includes several sub-datasets used for myoelectric pattern recognition research. One notable sub-dataset, referred to as BioPatRec DB2, comprises data from 11 subjects performing 17 different hand gestures.
- **CAPG-Myoelectric Database [28]:** a publicly available database that was collected from 23 intact subjects. It consists of 3 sub-database, which are DB-a, DB-b, and DB-c. Each sub-database focuses on different types of hand or finger movements. DB-a and DB-b Contain 8 hand gestures, both isometric (static) and isotonic (dynamic). The gestures were obtained from 18 of the 23 subjects in DB-a, and from 10 of the 23 subjects in DB-b. DB-c focuses on

12 basic finger movements, recorded from 10 of the 23 subjects. Each gesture was performed for 3–10s and repeated 10 times, and between each gesture a resting posture was held for 7s.

- UCIGesture [29]: another open to the public dataset that contain a raw EMG data from 36 subjects performing 7 basic hand gestures with two repetition in each. The gestures lasted for 3s with 3s of rest between them.

1.4 Conclusion

This chapter gave an overview the muscles of the upper limb, from their structural physiology to the neural mechanisms that command them. In addition, a study of electromyography was presented, covering an understanding of how EMG detects the bioelectric signals of muscle activity, using instruments such as gelled and dry surface electrodes or needle and fine-wire inserted electrodes, while addressing confounding factors like cross-talk, movement artifact, and intrinsic noise. Such concepts provides a foundation for understanding the mechanisms behind hand gesture and therefore contribute improving their automatic recognition. The chapter concludes by reviewing the most widely used EMG datasets for hand gesture recognition, highlighting their characteristics, differences, and relevance to practical applications.

Chapter 2 Deep Learning for EMG-based Hand Gesture Recognition Systems

2.1 Introduction

The chapter focuses on the Transformer architecture and its applications to EMG-based hand gesture recognition. A detailed overview of traditional deep learning models, including convolutional and recurrent neural networks (CNNs and RNNs) can be found in Appendix A . The chapter concludes with a comprehensive review of state-of-the-art research in this domain is provided to illustrate the current landscape of the deep learning for EMG-based hand gesture recognition.

2.2 The Transformer Architecture

The paper titled “Attention Is All You Need” [30] introduced a groundbreaking new architecture: the Transformer. As the title cleverly suggests, this model relies entirely on self attention operation. Like LSTMs, the Transformer is designed to transform one sequence into another, utilizing both an encoder and a decoder. However, what truly sets it apart from earlier sequence-to-sequence models is its complete abandonment of Recurrent Networks (like GRU or LSTM). Historically, Recurrent Neural Networks were considered the go-to solution for capturing temporal dependencies within sequences. Yet, the team behind the Transformer demonstrated that an architecture built solely on self attention operations, without any RNNs, could actually outperform previous models on translation and other tasks. This innovation paved the way for significant advancements in natural language processing, including models like BERT [31]. Hereafter, we review the key component forming the Transformer deep neural network architecture.

2.2.1 Encoder-Decoder Structure

The Transformer model architecture is fundamentally built upon an encoder-decoder structure, as illustrated in Figure 2.1. The encoder, positioned on the left, and the decoder, on the right, are each composed of N_x identical stacked modules. Each module primarily consists of Multi-Head Attention and Feed-Forward layers. Inputs and outputs (target sentences) are initially converted into an n -dimensional embedded space, as raw strings cannot be directly processed.

A crucial aspect of the model is the incorporation of positional encoding. Unlike

recurrent networks that inherently process sequences sequentially and thus maintain order, Transformers process inputs in parallel. Therefore, to account for the sequential nature of data, each word or element in the sequence is assigned a unique positional encoding. These positional encodings are additively combined with the embedded representation (an n -dimensional vector) of each word, providing the model with information about the relative or absolute position of tokens within the sequence.

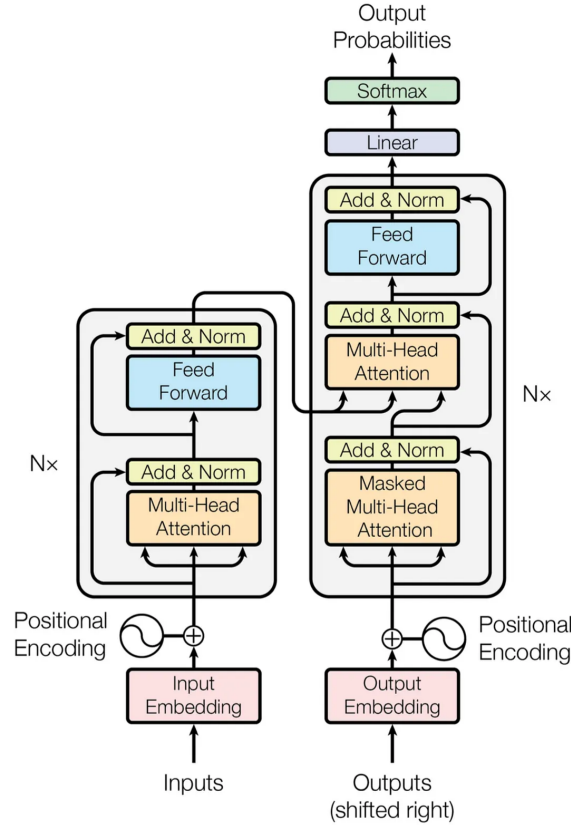


Figure 2.1: The Transformer Model Architecture [30]

2.2.2 Multi-Head Attention

The Multi-Head Attention mechanism, a core component of the Transformer, is depicted in Figure 2.2. This mechanism allows the model to selectively focus on different parts of the input sequence when processing each element.

The left panel of Figure 2.2 illustrates the Scaled Dot-Product Attention, defined by the following equation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.1)$$

Here, Q represents a matrix containing the query vector (typically the vector representation of a single word in the sequence), K contains the key vectors (representations of all words in the sequence), and V contains the value vectors (also

representations of all words in the sequence). In the encoder and decoder's self-attention modules, V consists of the same word sequence as Q . However, in the encoder-decoder attention module, V corresponds to the encoder's output sequence, while Q (and K) originates from the decoder's input sequence.

To elucidate, the output values in V are weighted and summed, where the attention weights $\mathbf{a}$ are defined by:

$$\mathbf{a} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \quad (2.2)$$

these weights $\mathbf{a}$ quantify how much each word in the sequence (represented by Q) is influenced by all other words in the sequence (represented by K). The softmax function normalizes these weights to a distribution between 0 and 1. These attention weights are then applied to the values in V .

The right panel of Figure 2.2 demonstrates how this attention mechanism is parallelized into multiple heads forming the Multi-Head Attention. This parallelization involves repeating the attention computation multiple times with different, learned linear projections of Q , K , and V (achieved by multiplying Q , K , and V with distinct weight matrices W^Q, W^K, W^V for each head). This allows the model to learn different representation subspaces for Q , K , and V , enabling it to focus on various aspects of the input at different positions and contributing to the model's overall representational capacity [30].

The matrices Q , K , and V are specifically generated for each attention module based on its position within the architecture of Transformer (encoder, decoder, or encoder-decoder connection). This differentiation ensures that the attention mechanism correctly attends to either the entire encoder input sequence or the relevant part of the decoder input sequence. The multi-head attention module connecting the encoder and decoder is crucial for integrating information from the encoder's output with the decoder's current processing stage.

Following the multi-head attention in both the encoder and decoder, a point-wise feed-forward layer is applied. This small feed-forward network has identical parameters for each position in the sequence, essentially performing a separate, identical linear transformation on each element of the given sequence [30].



(a) Scaled Dot-Product Attention

(b) Multi-Head Attention

Figure 2.2: Multi-Head Attention, Consisting of Several Attention Layers Running in Parallel [30]

2.2.3 Transformer for Time Series

Using the Transformer architecture for time series can be difficult. Time series data shares some characteristics with natural language, but it differs in critical ways. In language, you can express the same idea using vastly different words or sentence orders. Once a language-based Transformer such as the vanilla one has been trained on a language, it can understand the relationship between words, so when you represent an idea in two different inputs, the Transformer can still infer similar underlying representations. Time series, however, requires a strict ordering, as the sequence of data points is significantly more important. This presents a challenge for using Transformers for time series [32].

Despite the difficulties in using the Transformer for time series, it has the capability to improve the classification of time series data through self-attention mechanisms that effectively capture both global and local temporal relationships, thus enabling discriminative feature extraction from complex multivariate data. Compared to recurrent models that process the data sequentially, the use of positional encodings by the Transformer preserves the temporal relationship between the input data, an important consideration for the identification of fine class-specific structures. Recent research like ShapeFormer [33] suggests that combining generic feature extraction with class-specific shapelet discovery employing transformer modules allows these models to achieve better classification performance on multivariate time series classification tasks. In addition, the use of Transformers allows the parallel computation of the whole sequence during training to produce greater efficiency and better representation learning for classification tasks that are dominated by long-range dependencies. These improvements suggest that with certain adaptations like improved positional encoding and classification heads, the use of Transformer models is a strong rival to the use of traditional architectures for time series classification.

2.2.4 The Quadratic Complexity Issue

According to Zhou et al. [34], the quadratic complexity of the Transformer’s self-attention mechanism presents a significant challenge when dealing with long time-series data, particularly in applications like hand gesture recognition using EMG signals. A standard self-attention layer exhibits an $O(L^2)$ space and time complexity, where L represents the sequence length. This inefficiency becomes a notable bottleneck, especially when multiple Transformer layers are stacked, leading to an overall memory usage of $O(J \cdot L^2)$ (where J is the number of layers) and severely hindering scalability. Furthermore, the typical dynamic, sequence-by-sequence decoding process employed by Transformers can result in slower inference speeds compared to traditional RNN-based models. These inherent computational and memory demands make the application of standard Transformers to very long time-series sequences, such as those often found in EMG signal analysis, computationally intensive and resource-demanding for classification tasks.

2.3 Related Works

There are several approaches to identify hand gestures from EMG signals. While most of them are based on the deep learning architectures that we introduced earlier. Some have introduced hybrid models that combine multiple architectural paradigms, some have proposed customized versions with task-specific modifications tailored to the unique challenges of biosignal processing. The following reviewed works considered to be the most interesting in the classification of hand gestures using EMG signals:

- **Transformer-based Hand Gesture Recognition From Instantaneous to Fused Neural Decomposition of High-Density EMG Signals [35] :**

Montazerin et al. [35] suggest a Compact Transformer-based Hand Gesture Recognition (CT-HGR) system in which a Vision Transformer is directly used on HD-sEMG signals and self-attention to learn the spatial and temporal features at the same time without handcrafted pre-processing or transfer learning. CT-HGR splits the HD-sEMG data into patches across sliding windows (31.25–250 ms) and performs real-time single-frame instantaneous classification from single-frame EMG images. A hybrid version also includes macroscopic HD-sEMG images with microscopic motor unit spike trains obtained through blind source separation to include neural drive data. Evaluated across 65 isometric hand gestures, resulting in 66 total movements as one gesture was repeated from 19 subjects (128 electrodes), CT-HGR achieved average accuracy from 86.2 % (32 electrodes, 31.25 ms) to 91.98 % (128 electrodes, 250 ms), and 89.13 % for one-frame recognition, and performs better than 3D CNN and SVM models.

- **EMGHandNet: A Hybrid CNN and Bi-LSTM Architecture for Hand Activity Classification Using Surface EMG Signals [36]:** "EMGHandNet: A Hybrid CNN and Bi-LSTM Architecture for Hand Activity Classification Using Surface EMG Signals" introduces a new deep learning strategy, EMGHandNet, for hand activity recognition from surface electromyography signals. The authors design a hybrid architecture that uses 1D convolutional neural networks (CNNs) to learn inter-channel spatial patterns and bidirectional long short-term memory (Bi-LSTM) layers to learn bidirectional temporal patterns to counteract spatial sparseness and temporal dynamics of sEMG signals. Evaluated on five benchmark databases (NinaPro DB1, DB2, DB4, BioPatRec DB2, and UCI Gesture, EMGHandNet achieved best-in-class performances at 95.77%, 95.9%, 91.65%, 91.29%, and 98.33%, respectively, which outperformed prior models up to 18.65% margins. Key innovations are end-to-end learning of inter-channel and temporal patterns, optimal input segmentation schemes, and bespoke pre-processing steps (e.g., wavelet denoising). The effectiveness of the model resides in its ability to learn short-term spatial correlations using CNNs and bidirectional temporal patterns using Bi-LSTMs and is hence very well-suited for prosthetic control, exoskeleton systems, and human-machine interaction applications. The study also demonstrates the necessity for dataset-specific pre-processing and indicates better computational

efficiency at higher complexity compared to simple models. Potential future directions are time-frequency representation exploration and hardware implementation to address high latencies.

- **LST-EMG-Net: Long Short-Term Transformer Feature Fusion Network for sEMG Gesture Recognition [37]:**

The "LST-EMG-Net: Long Short-Term Transformer Feature Fusion Network for sEMG Gesture Recognition" manuscript proposes a new Transformer-based framework for reducing the gap between complexities and lengths of sEMG characteristics. Recognizing that more complex movements require longer signal segments to maintain steady feature extraction and easy movements require shorter segments. Authors propose LST-EMG-Net that makes use of both dual long- and short-term encoders. The long-term encoder is based on temporal relationships among longer segments for complex movements and the short-term encoder is based on short segments for easy movements. Multi-scale feature integration is done by a feature cross-attention mechanism and an adaptively weighted channel attention mechanism weighs sEMG channels. To overcome the sparseness of the data, synthetic signal processing is employed to synthesize samples randomly by randomly selecting windows and time-delaying enhancements. Evaluated on sparse (Ninapro DB2, DB5, CapgMyo DB-c) databases, LST-EMG-Net achieves 81.47%, 88.24%, and 98.80% top-performing accuracy on all databases compared to MSCNN, BiTCN, and TEMG models. Ablations establish the contribution of all components, and robustness and inter-subject accuracy variation reduction are achieved by the improved robust model. The efficiency (6.47 ms inference time) and robustness of the framework make the framework the best choice for application to prosthetic control, stroke rehabilitation, and human-computer interaction.

- **Hand Gesture Recognition Using Temporal Convolution and Attention Mechanism [38]:**

The paper introduces a new Temporal Convolutions-based Hand Gesture Recognition (TC-HGR) approach for hand gesture classification from sparse multi-channel surface electromyogram (sEMG) signals. To mitigate structural complexity and high computation issues of conventional deep neural networks (DNNs), the TC-HGR utilizes dilated temporal convolutions and self-attention to perform effective temporal feature extraction and informative regions of the signal to attend to. The architecture achieves accuracies of 81.65% (window width 300ms) and 80.72% (window width 200ms) on the Ninapro DB2 database (40 subjects, 17 gestures), and reduces trainable parameters by $11.9\times$ compared to existing state-of-the-art LSTM-based methods. The main contributions are non-overlapping patching of the signals to minimize computational complexity, residual connections to regularize training, and exploitation of exponential dilation factors to maximize the receptive area. The lightweight architecture design (e.g., 94,965 parameters for 80.72% accuracy) holds significant potential for prosthetic control in real time, bridging the gap between research and clinical translation at the expense of accuracy.

- **Surface EMG based Hand Gesture Recognition Using Hybrid Deep Learning Networks [39]:**

The authors initially notch a low bar using a simple one-dimensional CNN on raw Ninapro signals, and score low accuracies ($\approx 52\%$ on DB2, 46% on DB3). They then show that normalization, windowed statistics, and recurrence-plot transformation substantially improve the input quality. Afterward, they incrementally introduce more sophisticated models comprising a more sophisticated 1-D CNN, an LSTM, 1+1 and 3+3 hybrid C-RNNs that score up to 85.3% (HAR), 57.1% (DB2), and 75.9% (DB5), and further tuning lifts DB2/DB5 to $63.7\%/83.6\%$. Finally, an attention-based bidirectional ConvGRU using both past and future correlation between signals and weights intra-channel features outperforms prior art on DB5 at 88.7% . To address amputee-subject variability, they employed transfer learning pretrained on 20 intact subjects and fine-tuned on just two amputees from NinaproDB7 to accelerate convergence. They achieved 76.2% accuracy. To address data scarcity, They employed a DC-GAN to synthesize realistic sEMG signals; combined with native data, these clones enhance robustness of models for limited-data conditions.

- **Deep Learning for Electromyographic Hand Gesture Signal Classification Using Transfer Learning: [40]:** Deep learning for hand gesture recognition from sEMG remains data hungry and is addressed by this paper, which deploys a transfer-learning strategy that pools data from many users to learn informative generalizable features and then fine-tunes to new users using many fewer instances. Leveraging two Myo-Armband datasets (19 and 17 healthy subjects), and making use of a third one (10 volunteers of NinaPro), the authors introduce three ConvNet models that operate on one of the three input modalities: continuous wavelet transforms (CWT), or spectrograms (STFT), or raw EMG, and show that pre-training from the first dataset consistently improves the remaining ones. Offline testing achieves 98.31% for the ConvNet from CWT for 17 users and seven hand-gesture classes and 68.98% for the ConvNet for 18 classes and 10 users from raw EMG. The work also introduces a new 36-subject sEMG dataset and the first application of CWT-based representations for this application. Finally, an eight-participant real-world use case shows that immediate feedback facilitates user adaptation of muscle activation and compensates for accuracy degradation over fourteen days without requiring adjustment to the model.

Table 2.1: Summary of the EMG Hand-gestures Classification Studies.

Study	Database	Gesture & Subject	Model	Accuracy (%)
Montazerin et al. [35]	HD-sEMG	66 gestures, 20 subjects	Transformer (CT-HGR)	86.2 (32 electrodes, 31.25 ms)
				91.98 (128 electrodes, 250 ms)
				89.13 (one-frame)
EMGHandNet [36]	NinaPro DB1	52 gestures, 27 subjects	CNN + Bi-LSTM	95.77
	NinaPro DB2	49 gestures, 40 subjects		95.9
	NinaPro DB4	52 gestures, 10 subjects		91.65
	BioPatRec DB2	26 gestures, 17 subjects		91.29
	UCI Gesture	6 gestures, 36 subjects		98.33
LST-EMG-Net [37]	NinaPro DB2	Exercise B, 17 gestures, 40 subjects	LST-EMG-Net (Long Short-Term Transformer Feature Fusion Network)	81.47
	NinaPro DB5	Exercise C, 18 gestures, 10 subjects		88.24
	CapgMyo DB-c	12 gestures, 10 subjects		98.80
TC-HGR [38]	NinaPro DB2	Exercise B, 17 gestures, 40 subjects	Temporal Convolutions + Self-Attention	81.65 (300ms)
				80.72 (200ms)
Hybrid Deep Learning Networks [39]	HAR	6 gestures, 30 subjects	Attention-based Bi-ConvGRU Network bidirectional ConvGRU	85.3
	NinaPro DB2	18 gestures, 40 subjects		63.7
	NinaPro DB5	18 gestures, 10 subjects		88.7
	NinaPro DB7	18 gestures, 22 subjects (20 intact, 2 amputees)	Transfer Learning	76.2
EMG Transfer-Learning Network[40]	MyoArmband	7 gestures, 17 subjects	ConvNet	98.31 (CWT)
	NinaPro	18 gestures, 10 subjects		68.98 (raw EMG)

2.4 Conclusion

This chapter examined the Transformer architecture and its application to EMG-based hand gesture recognition, highlighting its unique advantages in modeling of relationships through attention mechanism. It becomes apparent that the field is leaning towards the hybrid models, since combining different techniques can overcome the limitations of individual approaches. Despite the progress, problems like generalization still remain due to some constraints including the anatomical differences between users. Therefore, future research should focus on solving these problems so we can move closer to creating intuitive and reliable gesture recognition systems that can significantly improve the user quality of life.

Chapter 3 Data Preparation and Proposed Model Architecture

3.1 Introduction

To have an accurate and robust hand gesture recognition system the preparation of the raw data should be taken carefully in consideration, along with the model architectural design which must be capable of interpreting complex physiological signals. In this chapter, we first begin by going through the NinaPro dataset which we will be using in this project. With this dataset, we present the pre-processing pipeline that we used, starting from train/test splitting, through sliding window segmentation, z-score normalization, and sequential bandpass and notch filtering to denoise the data.

Following data preparation section, we introduce our model architecture and its components, that consists of four main components: a convolutional feature extractor, Time2Vec positional encoding, transformer encoder block with multi-head attention mechanisms, and a classification head to get the predictions. Each component is described in detail, showing its role in the overall architecture. Figure 3.1 provides an overview of our methodology.

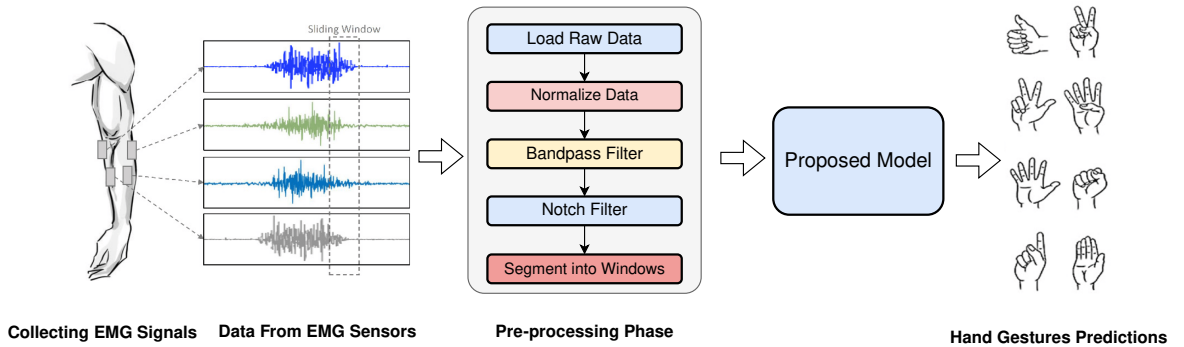


Figure 3.1: End-to-end System Overview.

3.2 NinaPro Dataset

NinaPro Dataset is the largest available hand gesture EMG dataset. A subset of NinaPro is selected for the evaluation of our part. It is an open access database which includes various kinds of measurements, it is designed to advance research

in machine learning for human, robotic, and prosthetic hand functions. NinaPro consists of 10 datasets, labeled DB1 to DB10, and in total they include over 180 data acquisitions from intact subjects and transradial hand amputees. NinaPro provides a variety of signals such as electromyography, kinematic, inertial, clinical, neurocognitive and eye-hand coordination data.

For the purposes of our study, we have chosen to use the second database (DB2) of NinaPro as it has the largest number of subjects. For DB2, each movement is repeated 6 times for a duration of 5s each and with 3s of rest between them. All 49 movements were recorded at a frequency of 2KHz, and separated on three MATLAB files, where each file represent exercises that have a set of gestures to perform. The exercises are:

- Basic movements of the fingers.
- Grasping and functional movements.
- Force patterns. [41].

Table 3.1: Exercises with number of gestures available [41].

Label in the File	No. of Gestures Performed	Exercise
B	17	Basic movements of the fingers
C	23	Grasping and functional movements
D	9	Force patterns

For each exercise and for each subject, its MATLAB file contains various synchronized variables [41]. The variables included in the MATLAB files are:

- **subject** (1 column): Subject number.
- **exercise** (1 column): Exercise label (A, B, or C).
- **acc** (36 columns): three-axes accelerometers of the 12 electrodes.
- **emg** (12 columns): Raw EMG signals recorded from the 12 electrodes.
- **glove** (22 columns): Uncalibrated readings from the 22 CyberGlove sensors.
- **stimulus** (1 column): Movement repeated as instructed to the subject.
- **restimulus** (1 column): Refined movement repeated by the subject to match the actual performed gesture.
- **repetition** (1 column): indications of the start and the end of each repetition based on stimulus.
- **rerepetition** (1 column): Refined indications of the start and the end of each repetition based on restimulus.

- **force** (6 columns): Force measurements collected during the recordings (only for force-pattern gestures in File C ,which is the third exercise).
- **forcecal** (2×6 values): The force sensors calibration values, corresponding to the minimal and the maximal force, which is available only for force-pattern gestures in File C , which is the third exercise.

DB1 to DB5 are the most widely used in research. Table 3.2 compares these five databases:

Table 3.2: Comparison of NinaPro DB1 to DB5 Datasets

[41]

Name	No. Subjects	No. Gestures	Sampling Freq.	No. Repetitions
DB1	27 (intact)	52	2 kHz	10
DB2	40 (intact)	49	2 kHz	6
DB3	11 (amputees)	49	2 kHz	10
DB4	10 (intact)	52	2 kHz	6
DB5	10 (intact)	52	200 Hz	6

All the databases focus on both classification and regression tasks in myoelectric control, DB3 uniquely targets amputees, which is crucial for prosthesis research. With 27 subjects in DB1 and 40 in DB2, these datasets provide a robust sample size for algorithm training and comparison, making them widely used for benchmarking. There is a slight difference in the sets of gestures recorded. The number of repetitions also varies. The sampling frequency differs among the databases because of the fact that the EMG acquisition hardware differs: DB1 uses Otto Bock electrodes, DB2 and DB3 use Delsys Trigno (which is known for high-quality recordings), DB4 uses Cometa electrodes, and DB5 employs Thalmic Myo Armbands [41].

3.2.1 Important Considerations for the NinaPro Dataset

A recent assessment of NinaPro dataset highlights critical considerations for researchers [42]:

- **Signal Quality Across NinaPro Datasets**
 - Most datasets had acceptable signal quality, but some trials contained excessive noise.
 - Filters were effective in suppressing low- and high-frequency noise, as well as power line interference (50 Hz).
 - Motion artifacts (less than 20 Hz) were minimized due to the experimental protocol (static limb position).
 - DB5 had the lowest signal quality due to its low sampling rate (200 Hz), which reduces the discriminative power of EMG features.

- Some trials had very low SNR (less than 1.8 dB), indicating severe noise contamination that could distort classification results.

- **Effect of Signal Quality on Classification Performance**

- SNR and classification accuracy showed a positive correlation ($r = 0.35$, p less than 0.05).
- Higher SNR led to better classification performance, confirming that cleaner signals improve gesture recognition.
- The writing tripod grasp gesture had the lowest SNR (approximately 8.3 dB), suggesting that certain gestures are more prone to noise

- **Issues with Data Labeling** Some EMG signals were incorrectly labeled, affecting classification accuracy. Examples of mislabeling:

- Missed contractions were not accounted for in the labels.
- Delays in muscle activation caused timing mismatches between labels and actual gestures.
- False no-motion labels: Some signals were active but labeled as rest, leading to confusion in classification.

Some data were ignored due to missing of recorded signals. For example, channels 9 and 10 in DB6 were ignored. Subject 6 and subject 7 from DB3 were excluded due to missing raw signals.

3.2.2 Gestures Selection

In this work, our attention is exclusively directed toward the very first category of hand movements: fundamental gestures involving individual finger motions and wrist articulations. From this set, we have identified and selected fifteen distinct gestures (see Table 3.3) for the purpose of algorithmic classification. To assemble our training and evaluation dataset, we begin by retrieving the raw electromyographic signals stored in the EMG field. These continuous voltage traces capture the muscular activity associated with each gesture. Next, we consult the restimulus labels, which have been refined by an automated alignment routine to ensure that each time segment is accurately tagged with the true gesture performed. The precise temporal boundaries of each repetition are then determined using the timestamps encoded in the rerepetition field, which marks both the onset and offset of every individual movement trial.

Table 3.3: List of Selected Hand Movements With Corresponding Gesture Images.

Label	Movement	Corresponding Image
1	Thumb up	
2	Extension of index and middle, flexion of the others	
3	Flexion of ring and little finger, extension of the others	
4	Thumb opposing base of little finger	
5	Abduction of all fingers	
6	6 Fingers flexed together in fist	
7	Pointing index	
8	Adduction of extended fingers	
9	Wrist supination (axis: middle finger)	
10	Wrist pronation (axis: little finger)	
11	Wrist flexion	
12	Wrist extension	
13	Wrist radial deviation	
14	Wrist ulnar deviation	
15	Wrist extension with closed hand	

3.3 Data Pre-processing

Several pre-processing steps and techniques were applied for the raw EMG signals data before feeding it to our model. These steps are essential to enhance data quality, reduce noise, and ensure consistency across samples. The pre-processing pipeline included data splitting, segmentation using sliding windows, normalization, and signal filtering to eliminate unwanted artifacts and denoise the raw signals. Each of these steps is detailed in the following subsections.

Data Split: To evaluate the model’s ability to generalize to unseen data, the dataset was split into training and testing sets. In the followed approach, for every individual subject the data was split based on the repetition:

- The 1st repetition along with the 2nd, 4th, and 6th was used for training.
- The 3rd and 5th were reserved for testing.

By applying this split separately to each subject, we ensure that the model is trained and evaluated on different executions of the same gestures, promoting better generalization across temporal variations.

Windowing: To segment the data into meaningful units, a sliding window approach was applied by dividing the signals into a fixed-length overlapping windows from the twelve EMG channels. Before applying the technique, the data was filtered to retain only the specified repetitions and gesture classes relevant to the experiment to ensure that the windowing process was applied only to the intended part of the dataset. After that, a window size of 300 samples (150 ms) was applied to the data to improve accuracy for gestures requiring intricate muscle activity, as it captures more stable signal segments.

To further enrich the temporal context and increase the number of training samples, a sliding window is moved 200 samples (100 ms) forward at each step. This corresponds to a 33% overlap between consecutive windows, ensuring that key transitions in muscle activation are not missed while keeping computational demands within real-time constraints.

Normalization: To eliminate scale differences across the twelve EMG channels and to improve model convergence, a z-score normalization was applied. Normalization parameters are estimated using only the training data and not on the test data to avoid data leakage. First, all samples belonging to the designated training repetitions were extracted and used to fit a StandardScaler. This scaler will calculate the mean μ_j and standard deviation σ_j for each channel based on the training data. After that the same transformation is applied to all the entire data (training and testing), where we subtracted the mean and divided by the standard deviation (calculated from the training set) for each channel. The z-score normalization formula is given by:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

where:

- x_{ij} is the original value of sample i from channel j .
- μ_j is the mean of channel j (calculated from the training set).
- σ_j is the standard deviation of channel j (also calculated from the training set).
- $\tilde{x}_{ij}$ is the normalized value.

This process ensures that all channels have zero mean and unit variance, helping the model learn more effectively.

Filtering: To clean the raw EMG signals, we applied two sequential digital filters: a bandpass Butterworth filter to confine the signal to its physiological bandwidth and reject motion artifacts and high-frequency noise, followed by a notch filter to eliminate power-line interference.

- **Bandpass Filter:** A bandpass (BPF) filter is a filter that allows signals within a specific frequency range (passband) to pass through while attenuating frequencies outside this range. It is formed by cascading a High Pass Filter (HPF) and a Low Pass Filter (LPF). The HPF blocks frequencies below its cutoff, while the LPF blocks frequencies above its cutoff [43]. The resulting frequency response of the BPF shows a clear passband bounded by the HPF and LPF cutoffs, as illustrated in Figure 3.2.

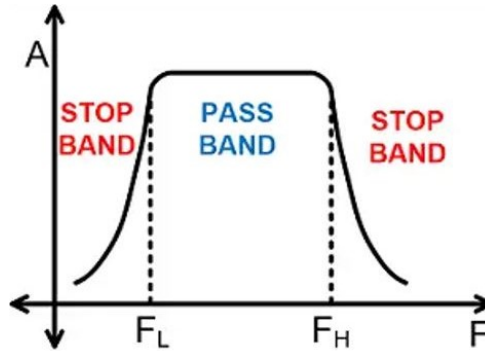


Figure 3.2: Frequency Response of Band Pass Filter [43].

According to [44], surface EMG signals are commonly filtered using a band-pass range of 20 to 500 Hz. Therefore, we implemented a zero-phase, 4th-order Butterworth bandpass filter with cutoff frequencies at 20 Hz (high-pass) and 500 Hz (low-pass). The Butterworth filter was chosen because it is widely recommended for EMG pre-processing [45].

- **Notch Filter:** A notch filter (also called a bandstop or band-reject filter) is a filter that blocks a narrow range of frequencies while allowing all other frequencies to pass. This filter acts as an opposite for the the bandpass filter. It blocks a specific frequency band (stopband) and passes all other frequencies (passbands). The cutoff frequencies of the notch filter define its stopband: the lower cutoff (f_ℓ) marks where attenuation begins, the higher cutoff (f_h) where it ends, and the bandwidth ($BW = f_h - f_\ell$) determines the rejected frequency range. Signals below f_ℓ and above f_h pass through, while frequencies within the stopband are blocked[46]. Figure 3.3 illustrates the basic characteristics of a notch filter.

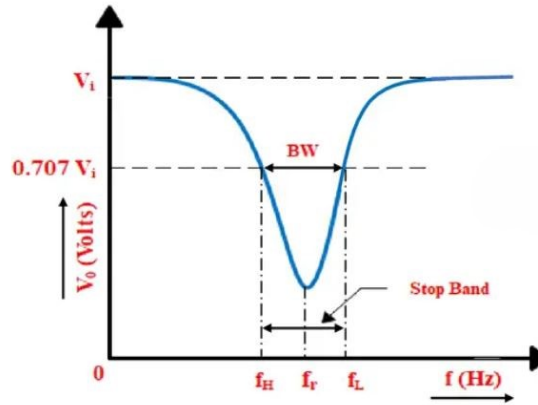


Figure 3.3: Notch Filter Characteristics [46].

Power-line interference (PLI) is a common and the most problematic source of ambient noise that typically arises due to differences in electrode impedance and the displacement currents within the subject’s body or connecting cables. Since it is linked to the electrical power grid, PLI manifests as a frequency centered around 50 Hz in Europe and 60 Hz in North America [47]. Given that the dataset used in this work was recorded in Europe [48], the interference is expected to occur at 50 Hz.

Taking this information into account, a notch filter centered at 50 Hz was implemented. The bandwidth (BW) of the notch filter was set to 10 Hz at a sampling frequency of 2000 Hz, which provides a reasonable balance between the sharpness of the notch and the amount of surrounding signal distortion. According to these parameters the filter starts attenuating slightly below 50 Hz, and fully rejects from about 45 Hz to 55 Hz.

3.4 Data visualization

In order to assess and illustrate the impact of pre-processing operation on the recorded electromyographic (EMG) signals, we will present a series of visual analysis in both the time and frequency domains. We aim to highlight how normalization, bandpass filtering, and notch filtering progressively enhance signal quality and preserve physiologically relevant features, and helped in denoising the EMG signals.

EMG Signals in Time Domain

Figure 3.4 demonstrates the time domain representation of EMG signal at different pre-processing stages. The figure shows only the first 1000 sample (0.5 seconds) from channel 0 of the first subject.

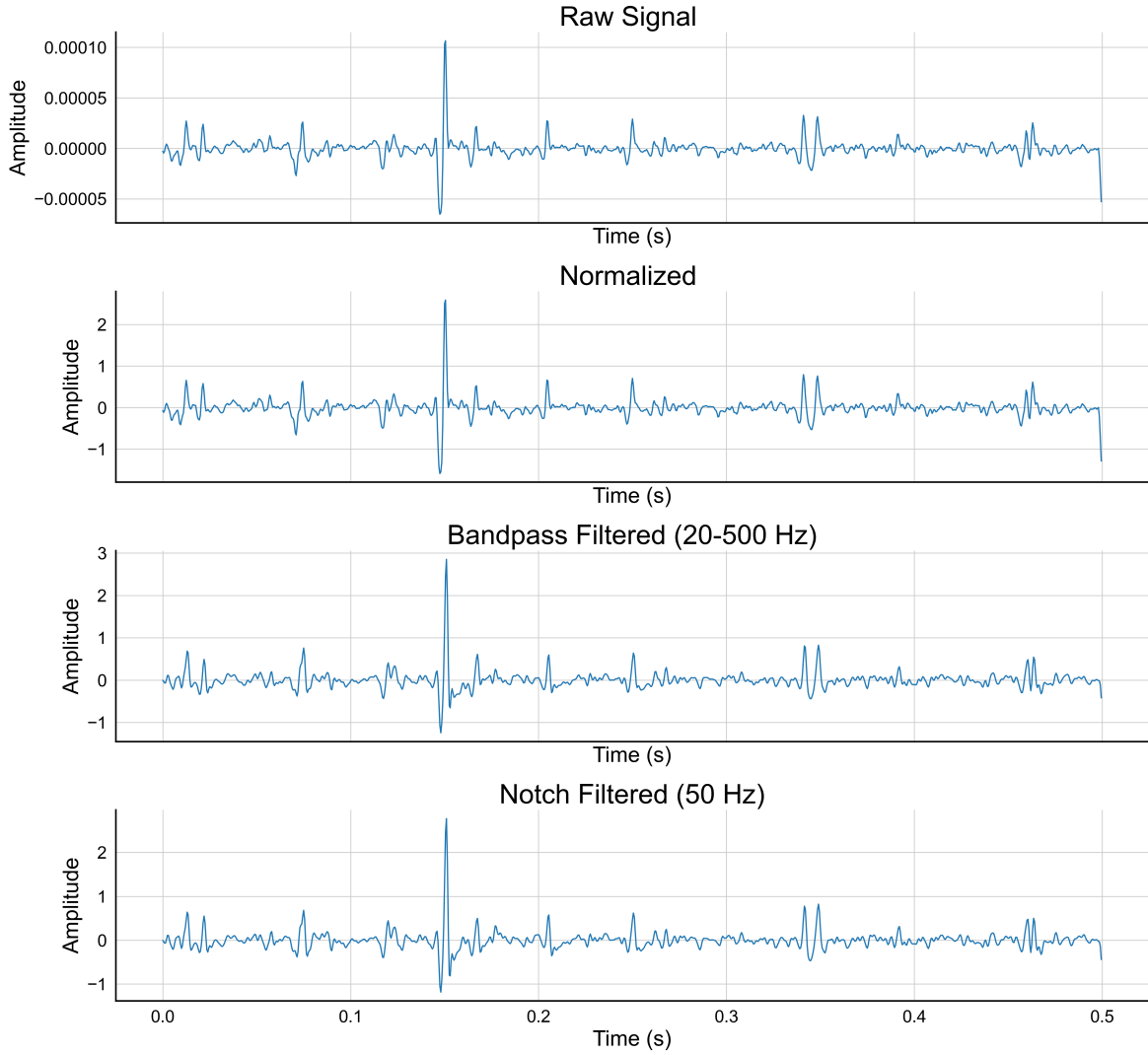


Figure 3.4: Effect of the Pre-processing Techniques on the EMG Signal in Time Domain.

It is clear that the normalization maintained the temporal pattern while standardizing the amplitude range from the raw signal, while the 20-500 Hz bandpass filter preserved meaningful muscle activation patterns while enhancing the clarity of spike events, the 50 Hz notch filter maintains the overall signal morphology while reducing noise. The effect of these techniques can be seen more clearly in the frequency domain.

EMG Signals in Frequency Domain

Figure 3.5 demonstrates the frequency domain representation of EMG signal at different pre-processing stages, with the frequency spanning from 0 to 600 Hz, and the resulting SNR is shown at each stage. This signal is also from channel 0 of the first subject.

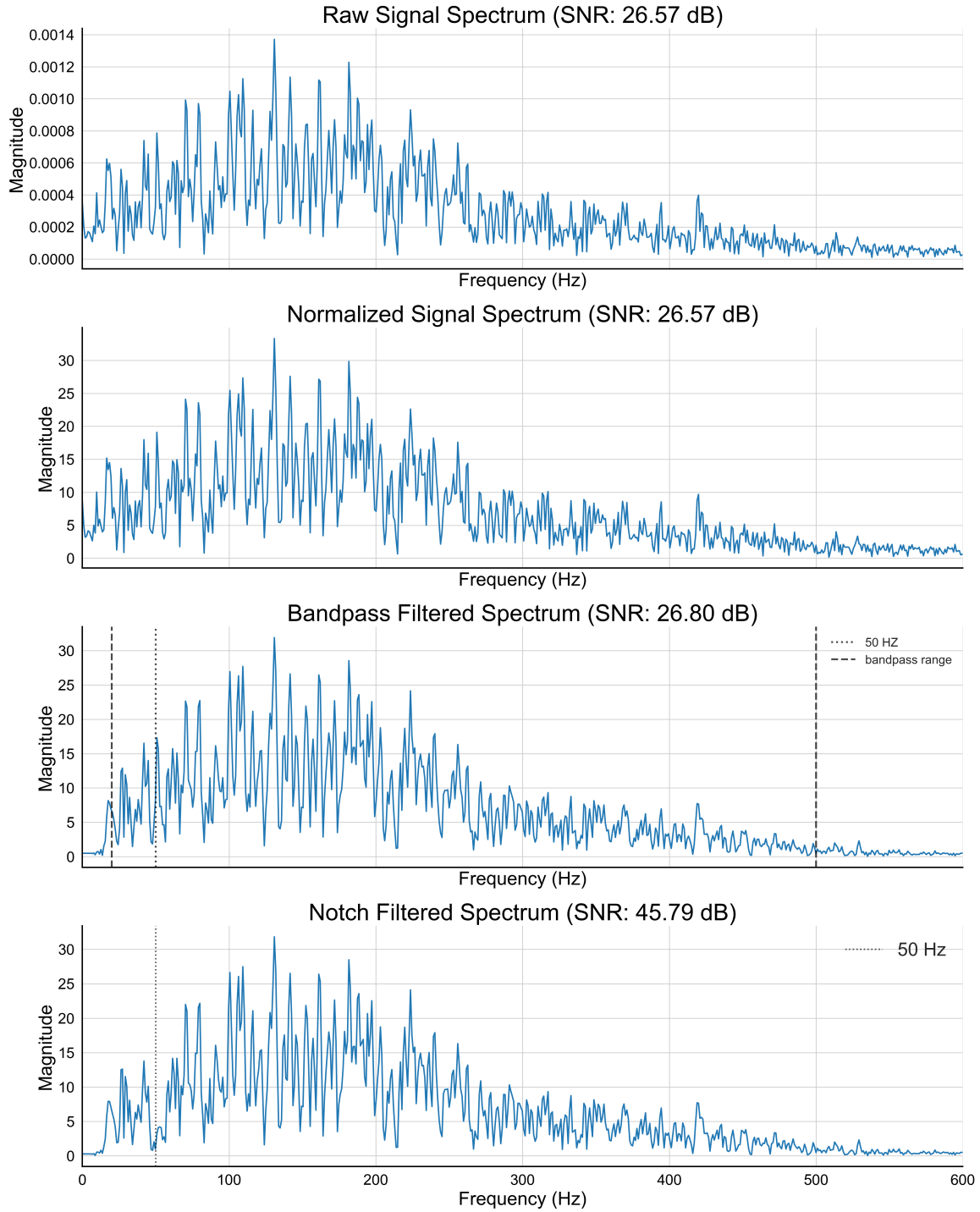


Figure 3.5: Effect of the Pre-processing Techniques on the EMG Signal in Frequency Domain.

The normalization step maintained the same frequency pattern as the raw signal but with scaled magnitude values, preserving the same signal-to-noise ratio (SNR) of the raw signal. The SNR was slightly improved at the bandpass filter stage which is a small improvement in denoising the signal. The filter has successfully attenuated frequencies outside the physiological bandwidth (20-500 Hz represented as dashed lines) and effectively removed low-frequency motion artifacts (below 20 Hz). The notch filter created a clear reduction in the signal power at precisely 50 Hz and dramatically improved the SNR by 18.99 dB as can be seen in Table 3.4.

Table 3.4: SNR Improvement Throughout EMG Signal Processing Pipeline.

Processing Stage	SNR (dB)	Improvement (dB)
Raw Signal	26.57	-
Normalized Signal	26.57	0.00
Bandpass Filtered Signal (20-500 Hz)	26.80	+0.23
Notch Filtered Signal (50 Hz)	45.79	+18.99

In order to clearly demonstrate the effect of the notch filter on the signal, a close-up look at the signal at the 50 Hz frequency is provided in Figure 3.6.

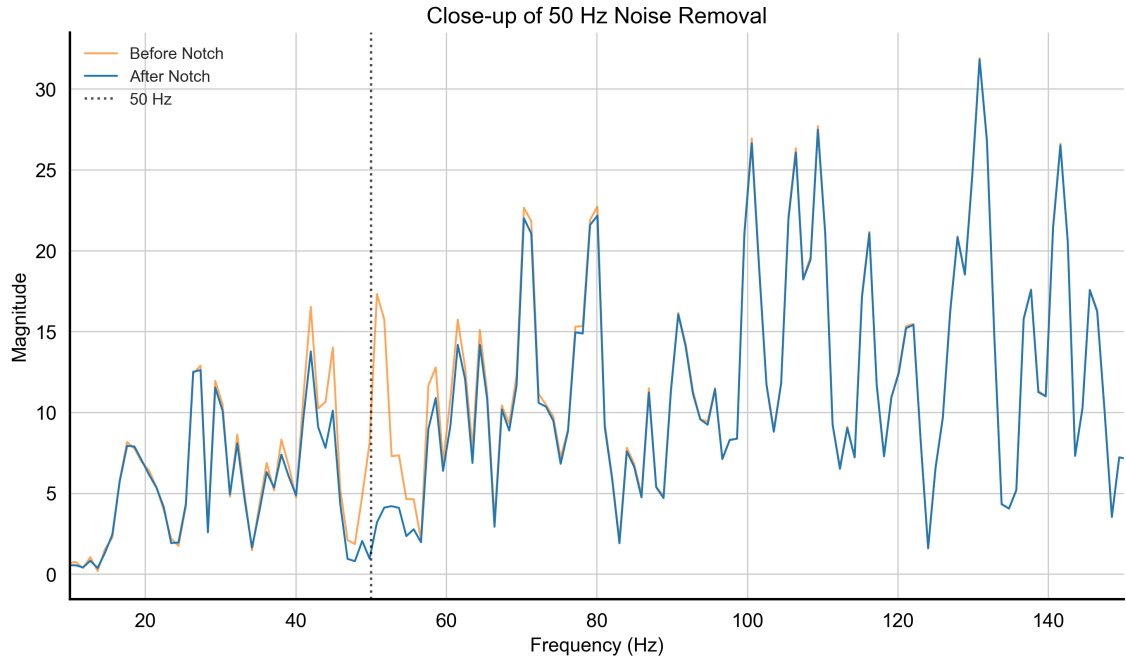


Figure 3.6: Effect of the 50 HZ Notch Filter on the EMG Signal.

3.5 Proposed Model Architecture

3.5.1 Overall Network Diagram Summary

Our model adapts the transformer architecture specifically for gesture recognition by using only the encoder component, eliminating the need for sequence generation.

Key innovations of the proposed architecture include Time2Vec positional encoding for better temporal pattern capture and convolutional feature extraction to reduce computational complexity while preserving essential patterns.

Classification typically requires mapping the input to a fixed set of labels, which can be efficiently done by adding a classification head on top of the encoder output. Using only the encoder reduces parameters and speeds up both training and inference, while still capturing all the long-range dependencies and inter-channel relationships necessary for gesture recognition with the help of the self-attention.

The overall flow of our model is as follows: the preprocessed input data first passes through a convolutional feature extractor to capture local patterns. Next, a positional encoding which is Time2Vec is added to provide time-dependent information to help our model understand temporal patterns. After that, the data is processed by a Transformer encoder block, consisting of multi-head attention along with normalization, and a feed-forward network with residual connections to enhance learning stability. Finally, the encoded representation is passed to a multilayer perceptron (MLP) classification head to produce the output probability distribution over the gesture classes. Figure 3.7 shows a high-level view of the model architecture.

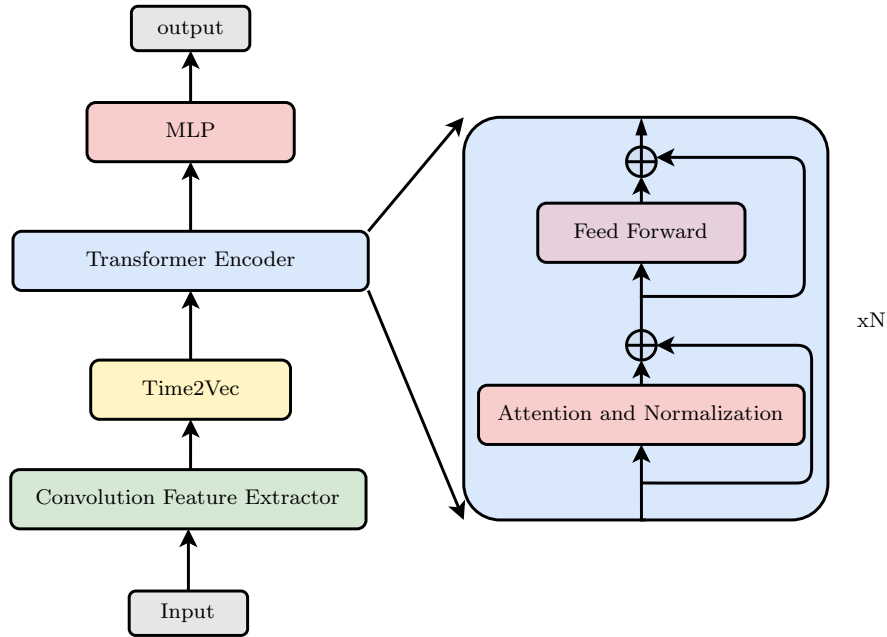


Figure 3.7: Overview of the Proposed Transformer Model.

3.5.2 Convolutional Feature Extractor

In this stage, we tried to efficiently capture local temporal patterns before passing data to the transformer encoder. Therefore, a 1D convolutional layer configured with 64 filters and a kernel size of 8, using ReLU activation to introduce non-linearity while capturing local temporal patterns in the input data. The convolutional layer extracts local features from the input signals while significantly reducing computational complexity compared to directly processing the input data with the trans-

former encoder. After that, a 1D MaxPooling layer configured with a pool size of 17 and stride of 9, further down-samples the sequence length, dramatically reducing the quadratic complexity of the subsequent self-attention operations. The down-sampling was required to decrease both the very high training and testing time that the model was taking, without sacrificing the model’s ability to detect relevant patterns. To prevent overfitting, a dropout layer was applied after the 1D Maxpooling layer. Next, batch normalization stabilizes the learning process by normalizing activations. Together, these operations create a more manageable representation that preserves essential patterns while enabling the transformer encoder to process the sequence more efficiently. Figure 3.8 presents a diagram summarizing this first component.



Figure 3.8: Convolutional Feature Extractor Layers.

3.5.3 Time2Vec Positional Encoding

The transformer relies on the self-attention mechanism, that allows it to process input sequences in parallel, and due to this parallelization, the model loses sense of the order of the sequence, because it does not understand which token appears before or after another token. Therefore, adding a positional encoding to solve this challenge is essential. The Time2Vec used here serves as a specialized positional encoding layer that allows our transformer model to understand temporal patterns and sequence order.

This positional encoding module can adapt to the specific temporal characteristics of gesture data, since it uses a learnable approach in contrast to other positional encodings. Time2vec transforms each time feature through two parallel pathways. The first is a linear projection that preserves the original temporal scale. While the second is a periodic component that captures cyclical patterns through sine activations. The linear component simply scales and shifts the input using trainable parameters, while the periodic component applies a learned frequency and phase shift using parameters followed by a sine activation. These components are then concatenated along the feature dimension, creating a rich representation that encodes both linear and cyclical temporal information.

Time2vec has advantages over the fixed positional encoding because it allows the model to learn the most relevant temporal representations for gesture recognition tasks. It is effective in capturing the different patterns present in human gestures.

3.5.4 Transformer Encoder

The transformer encoder block represents the core of our model architecture. It is designed to capture complex and long-range temporal dependencies in the data. Following the standard transformer design principles, our implementation consists of two main sub-blocks with residual connections. First, the data is processed with a Multi-Head Attention mechanism with an adjusted number of heads and head size.

This allows the model to focus on different parts of the input sequence simultaneously and learn relationships across various time steps. A Dropout layer, to prevent overfitting, followed by a Normalization layer, to stabilize training, are applied to the output of the Multi-Head Attention. Besides, a residual connection is added. Summary of the first sub-block is given in Figure 3.9.



Figure 3.9: Attention and Normalization Block Layers.

For the second part of the encoder block which is the feed forward sub-block, shown in Figure 3.10, a position-wise feed-forward network is implemented using two 1D convolutions with kernel size 1 (equivalent to position-wise fully connected layers in the original transformer paper). The first convolution uses ReLU activation to expand it to a higher dimension, while the second projects it back to the original dimensionality. Dropout is applied between these two convolutional layers. Layer normalization and another residual connection complete the encoder block, creating a robust architecture that effectively captures the temporal patterns necessary for accurate gesture recognition.

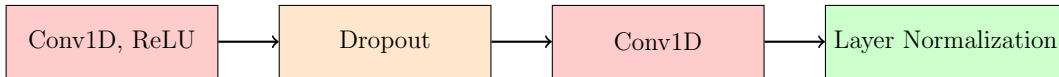


Figure 3.10: Feed Forward Block Layers

3.5.5 Classification Head

The classification head, shown in Figure 3.11, is the final block in our model. It is used to transform the rich sequential representations from the transformer encoder into class probabilities for gesture recognition. First, in order to capture the most salient features while being invariant to sequence length we introduced a 1D GlobalAveragePooling layer that will reduce the variable-length temporal features into a fixed-dimensional vector by averaging across the time dimension. This will also lead to a reduced number of parameters. After that, a dense layer with a ReLU activation, to introduce non-linearity, is added enabling the network to learn complex decision boundaries. Dropout is applied after this dense layer to prevent overfitting and improve generalization. Finally, a last dense layer outputs a probability distribution across all gesture classes using softmax activation, with each value representing the model's probability that the input belongs to a particular gesture class. This last block effectively uses the information captured by the transformer encoder and translates it into accurate gesture classifications.

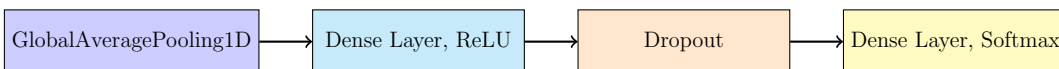


Figure 3.11: Classification Head

3.6 Conclusion

In summary, this chapter presented the foundation for our gesture recognition system, beginning with the dataset used in the study, to the data processing, ending at the model architecture design. Among the publicly available EMG datasets, the NinaPro DB2 dataset was selected due to high-quality recordings from a diverse group of subjects, and with our pre-processing pipeline we ensured that only high-quality denoised data were fed to the model. With this data preparation and the scalable model design we implemented, we are now ready to evaluate the performance of the model, which is the aim to the next chapter.

Chapter 4 Proposed Model Evaluation and Results

4.1 Introduction

Implementation, training, and evaluation of the proposed model for EMG-based hand gesture recognition will be presented in this chapter. Following the data preparation and the model architecture design described in Chapter 3, we outline the software tools, hyperparameter configurations and model optimization. A detailed list of the technologies used throughout the implementation can be found in B. We then evaluate the impact of our pre-processing pipeline and analyze the computational efficiency of the proposed model. Finally, we compare our results against other models tested on the same dataset to demonstrate our model performance capability.

4.2 Implementation Details

This section outlines the core software integral to our development and experimental workflow, including the main platforms and environments that facilitated the collaboration and large-scale computation. In addition, we describe the main libraries and frameworks used for data processing, model specification, training, evaluation, and visualization of results. These implementation choices outline a description of the environment in which our Transformer-based time-series classification pipeline was developed, trained, and tested.

4.2.1 Libraries

- **TensorFlow**¹: Open-source machine learning framework by Google, used to build and train deep learning models, including the Transformer architecture, custom Time2Vec layer, and Adam optimizer. Facilitated model compilation, evaluation, and predictions.
- **NumPy**²: Core library for numerical computations in Python. Supported array operations such as reshaping data, calculating mean and standard deviation (e.g., `mean`, `std`), and handling data structures for model input.

¹<https://www.tensorflow.org/>

²<https://numpy.org/>

- **Keras**³: High-level API within TensorFlow for defining neural network layers (e.g., `Conv1D`, `MultiHeadAttention`, `Dropout`), activations, regularizations (e.g., L2, dropout), and model construction. Simplified layer design and training workflows.
- **Scipy**⁴: Scientific computing library used for loading and saving MATLAB files (`loadmat`, `savemat`) to handle input data and store predictions.
- **Matplotlib**⁵: Plotting library for visualizing results, including confusion matrices (via `mlxtend.plotting`) and training metrics (accuracy and loss) over epochs. Generated figures saved as PNG files.
- **Scikit-learn**⁶: Machine learning library used for performance evaluation metrics, specifically `confusion_matrix` for analyzing classification results and `matthews_corrcoef` for computing the Matthews correlation coefficient (MCC).
- **MLxtend**⁷: Machine learning extensions library utilized for plotting normalized confusion matrices (`plot_confusion_matrix`) to visualize model performance across subjects.
- **Xlsxwriter**⁸: Python library for creating and writing Excel files. Used to generate a workbook (`Trans_rslt.xlsx`) storing performance metrics (e.g., accuracy, loss, MCC) for each subject.
- **Time**⁹: Standard Python library used to measure training and testing times (`time.time()`) for model evaluation.

4.3 Test Environment

The following subsection provides a detailed overview of the machine environments employed in our workflow, including pre-processing of the data, model training, and performance benchmarking. Machine 1 and 3 were primarily used for the pre-processing tasks, as well as for initial model training. Machine 2, which is equipped with CUDA acceleration, was used to significantly enhance the speed of deep learning experiments.

Machine 1

- **Processor:** Intel(R) Core(TM) i5-6300U CPU @ 2.40 GHz
- **RAM:** 8 GB

³<https://keras.io/>

⁴<https://scipy.org/>

⁵<https://matplotlib.org/>

⁶<https://scikit-learn.org/stable/>

⁷<https://rasbt.github.io/mlxtend/>

⁸<https://xlsxwriter.readthedocs.io/>

⁹<https://docs.python.org/3/library/time.html>

- **Storage:** 256 GB
- **Operating System:** Windows 10 Home Edition, 64-bit

Machine 2

- **Processor:** Intel(R) Core i7-6300U CPU @ 2.50 GHz
- **RAM:** 8 GB
- **Storage:** 512 GB
- **GPU:** NVIDIA GeForce 940MX
- **Operating System:** Windows 11 Home Edition, 64-bit

Machine 3

- **Processor:** Intel(R) Core i5-10310U CPU @ 1.70 GHz
- **RAM:** 8 GB
- **Storage:** 256 GB
- **Operating System:** Windows 10 Home Edition, 64-bit

4.4 Evaluation Metrics

- **Confusion Matrix:** The confusion matrix provides the number (and percentage) of predictions versus actual classes, with the four possible outcomes being True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). In binary classification (Table 4.1), the matrix captures the model's performance: TP (true positives), TN (true negatives), FP (false positives), and FN (false negatives). These values are used to compute several performance metrics: overall accuracy, precision (true positive rate), recall (sensitivity, i.e., the proportion of actual positives correctly classified), and specificity (true negative rate, i.e., the proportion of actual negatives correctly classified). Other measures such as F1-score and false positive rate are also derived from these basic values.

Table 4.1: Confusion Matrix of a Classification Problem.

	Positive	Negative
Positive	TP	FN
Negative	FP	TN

- **Overall Accuracy (Accuracy):** Overall accuracy indicates the rate of correctly classified predictions, expressing the rate of correctly classified predictions across the entire dataset. It easily adapts to multi-class classification problems.

The simplest way to evaluate the performance of a machine learning algorithm is to measure the classification accuracy, defined as the fraction of correct classifications (true positives and true negatives) relative to the total number of classifications. However, this metric can sometimes give deceptively high performance results.

Overall accuracy is given by the equation:

$$\text{ACC} = \frac{TP + TN}{TP + TN + FP + FN} \times 100.$$

..

- **Matthews Correlation Coefficient (MCC):** The Matthews correlation coefficient (MCC) measures the quality of binary (observed vs. predicted) classifications. It is regarded as a balanced metric, applicable even when classes differ greatly in size. MCC adds an extra dimension to performance evaluation and can also be extended to multi-class classification problems.

MCC yields a value between -1 and $+1$: a perfect classifier returns $+1$, random performance returns 0 , and total disagreement returns -1 .

The MCC is given by the equation:

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

4.5 Experimental Results and Discussion

In order to tune the hyperparameters of our model to reach its best performance, we will conduct a series of experiments to identify and fine tune the key hyperparameters of the model. Based on this set of series of experiments, we will analyze and discuss the results, updating each hyperparameter from its initial value to its optimal value. In addition, we will carry out supplementary experiments, such as evaluating the impact of our pre-processing pipeline to justify our design choices. Before that, we present the initial hyperparameter settings in Table 4.2 before we dive into the experiments:

Table 4.2: Initial Model Key Hyperparameters Configuration

Hyperparameter Category	Parameter	Value
Model Architecture	Transformer Blocks	1
	Head Size	32
	Number of Heads	1
	Feed-Forward Dimension	24
	MLP Units	[128]
Regularization	Transformer Dropout	0.25
	MLP Dropout	0.4
	Conv1D Dropout	0.3
	Bias Regularization	0.01
Optimization	Learning Rate	0.0001
	Batch Size	64
	Epochs	100

4.5.1 Hyperparameter Tuning Experiments

Number of Epochs: The number of training epochs significantly impacts model performance. Few epochs can lead to underfitting, while too many can cause overfitting to the training data. In order to set the optimal number of epochs for our model, we will conduct a number of experiments with different epoch values, including the initial value of 100 epochs. The results of the experiments are summarized in table 4.3.

Table 4.3: Number of Epochs Tuning Experiments Results

Metric (epochs)	50	100	150	200	250
Avg. Train Acc. (%)	78.95	86.56	90.29	92.61	94.30
Avg. Train Loss	0.5971	0.3796	0.2782	0.2176	0.1757
Avg. Test Acc. (%)	72.99	77.39	78.61	79.63	79.78
Avg. Test Loss	0.8043	0.6895	0.6868	0.6887	0.6885
Avg. MCC	0.7119	0.7579	0.7711	0.7819	0.7840

The average test accuracy is computed across all 40 subjects in our dataset. As it can be clearly seen in Figure 4.1, the **average test accuracy showed minimal improvement beyond 200 epochs**, while the average training accuracy continued to increase from 92.61 % at epoch 200 to 94.30 % at epoch 250, the test accuracy only slightly improved from 79.63 % to 79.78 %.

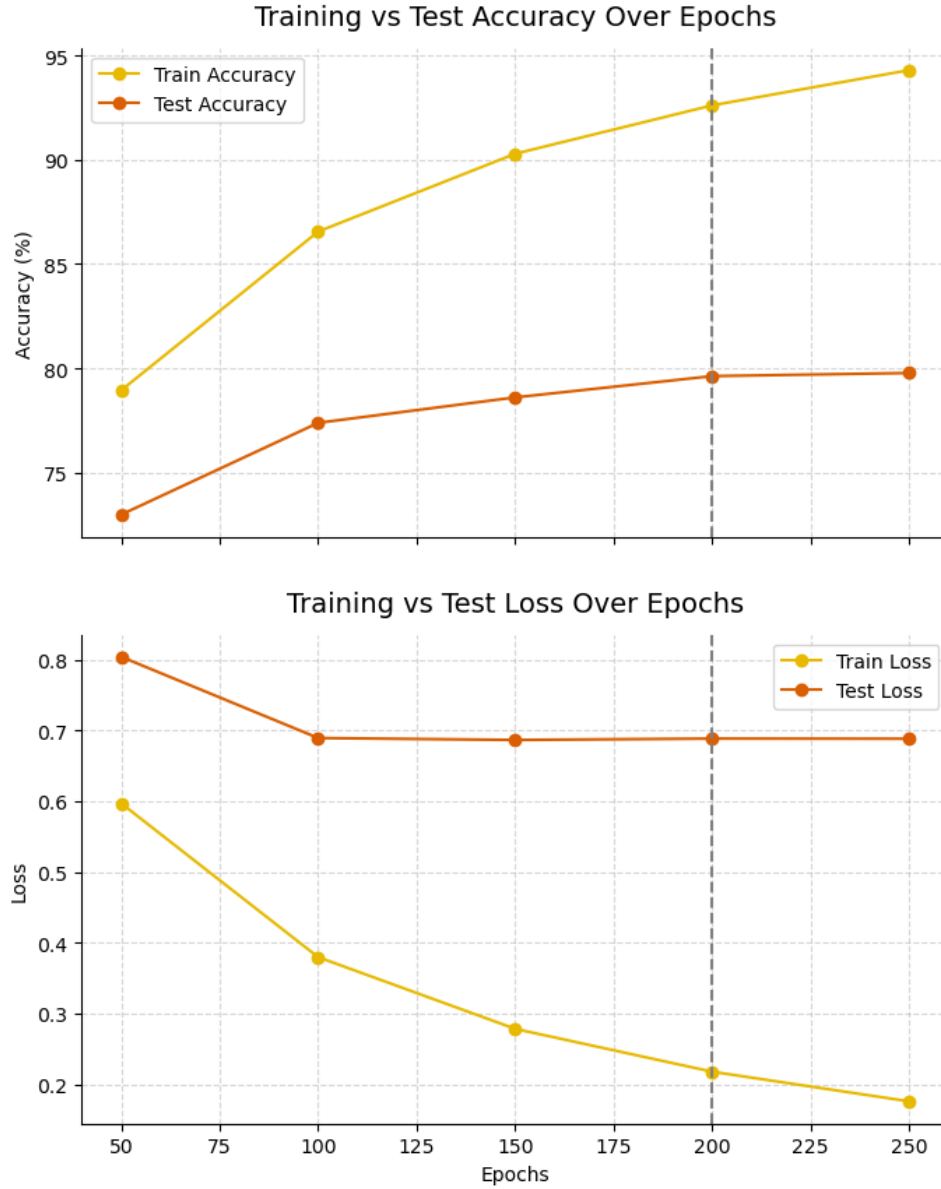


Figure 4.1: Accuracy and Loss Performance Over Epochs

Therefore, continuing training beyond 200 epochs may lead to overfitting, so **we will conduct the remaining experiments with 200 epochs.**

With the optimal training duration established at 200 epochs, we next investigate the fundamental optimization parameters that control how the model learns during each epoch.

Learning Rate and Batch Size: In this part of the hyperparameter tuning experiments, three different configurations will be tested to explore the critical hyperparameters of learning rate and batch size, which can significantly impact training speed and model performance.

- **Standard Approach (LR: 0.001, Batch Size: 64):** This is the default learning rate for Adam optimizer in most deep learning frameworks. The opti-

mizer can make reasonably aggressive updates while Adam’s momentum terms prevent instability. For many problems, this serves as an excellent starting point before fine-tuning. The gradient variance from batch size 64 works well with Adam. This combination is neither too aggressive nor too conservative.

- **Fast Learning (LR: 0.0005, Batch Size: 32):** The learning rate is between aggressive (0.001) and conservative (0.0001) and it converges faster than 0.0001 and it is more stable than 0.001. The batch size of 32 leads to more updates per epoch.
- **Stable Learning (LR: 0.0001, Batch Size: 128):** A large batch size was tested here since it acts like a regularizer, improving the model performance on unseen subjects leading to a better generalization. The same learning rate as the previous experiment (0.0001) was kept in order to get a maximum stabilization since a large batch and a small LR leads to a good stabilization.

These combinations did not include very small batches (e.g. 8, 16), as they may be too noisy for our architecture. We also did not include extreme learning rates (e.g. 0.01, 0.00001) to avoid instability or a very slow convergence. The results of the experiments can be observed in Table 4.4 below.

Table 4.4: Learning Rate and Batch Size Tuning Experiments Results

Metric (LR, BS)	(0.001, 64)	(0.0005, 32)	(0.0001, 128)
Avg. Train Acc. (%)	97.38	97.34	90.75
Avg. Train Loss	0.0900	0.0963	0.2705
Avg. Test Acc. (%)	82.21	81.96	78.45
Avg. Test Loss	0.8147	0.7631	0.6961
Avg. MCC	0.8095	0.8068	0.7693

The standard setup delivered the best test accuracy (82.21%) and highest MCC (0.8095). Fast is close behind (81.96 % test accuracy, 0.8068 MCC) and posts a slightly lower test loss (0.7631 vs. 0.8147), indicating that its noisier updates yield marginally better probability calibration. The stable setup achieved the lowest test loss (0.6961), which means it had a very smooth and well-calibrated predictions, but unfortunately it sacrificed the test accuracy (78.45 %) and MCC (0.7693) underscoring clear underfitting.

In the upcoming experiments, **we will adopt Standard setup (0.001, 64) for the best overall accuracy and MCC.**

Having optimized the learning dynamics, we now examine the core architectural parameters of the attention mechanism that determine the model representational capacity.

Number of Heads and Head Size: Attention mechanism configuration can impact the model performance. thus, four configurations will be tested by varying the number of attention heads and their sizes, from lightweight options to

high-capacity setups to observe the effect of these parameters on the model ability to capture relationships in the data.

- **Balanced Configuration (Head Size: 64, No. of Heads: 8):** Using 8 heads with a size of 64 results in 512 model dimension which is common in many Transformer variants (e.g. BERT). A 64 head size can be enough to capture complex relationships within each attention head, while the eight parallel heads can cover a variety of temporal patterns.
- **Lightweight Option (Head Size: 64, No. of Heads: 4):** To see if a complex attention is even needed and if a simpler one may be sufficient, we used a head size of 64 and decreased the number of heads from 8 to 4 compared to the last experiment anticipating that a simpler temporal patterns may perform better.
- **High-Capacity (Head Size: 128, No. of Heads: 12):** The setup includes a large head size of 128 with 12 heads. Due to the head size used, each head can learn very complex relationships, and more temporal patterns can be captured compared to the other two first combination due to the high number of heads used (12).
- **Efficient Learning (Head Size: 32, No. of Heads: 16):** The 32 head size used can force each head to focus on specific, distinct, and simple patterns. Building on this, a total of 16 heads is employed to allow each head to specialize in capturing specific features. The total model dimension is 512, same as the first combination.

The results of the experiments can be observed in Table 4.5 below.

Table 4.5: No. of Heads and Head Size Tuning Experiments Result

Metric (head size, No. of heads)	(64, 4)	(64, 8)	(32, 16)	(128, 12)
Avg. Train Acc. (%)	97.98	98.26	98.52	98.41
Avg. Train Loss	0.0724	0.0660	0.0594	0.0626
Avg. Test Acc. (%)	82.50	82.27	82.21	81.90
Avg. Test Loss	0.8065	0.8345	0.8643	0.8485
Avg. MCC	0.8124	0.8100	0.8095	0.8063

The Lightweight (64, 4) setup has the highest test accuracy (82.50 %) and highest MCC (0.8124), with test loss of 0.8065. The 64-dimensional head size seems to have enough capacity per head to capture key relationships, and the setup proved that using just four heads with that dimensionality is enough, since the performance of the Balanced (64, 8) setup that uses eight heads instead of four is nearly identical to Lightweight one, and slightly lower in terms of test accuracy (82.27 %) and MCC (0.8100). The High-Capacity (128, 12) presneted the lowest test accuracy (81.90 %) and MCC (0.8063) indicating that the over parameterization per head may encouraged the model memorization of the patterns, leading to a reduction

in classification performance on unseen trials. Interestingly, the Efficient (32, 16) setup recorded the lowest test loss (0.8643), though its accuracy and MCC were comparable to the Balanced configuration.

Based on these experiments, **we will adopt the lightweight setup (64, 4) as the default attention configuration for best average and MCC performance in the coming experiments.**

With the attention mechanism configured for optimal pattern recognition, we turn our focus to the dimensional aspects of the transformer internal processing layers.

Transformer Feed Forward and MLP Dimension: Another four different dimensional configurations will be tested on the transformer feed-forward network and the classification MLP head to explore the balance between transformer representation capacity and classification power.

- **Balanced Dimensions (MLP Dimension: 64, FF Dimension: 128):** The purpose of this setup was to make the FF dimension large to allow the transformer to create rich intermediate representations and capture complex patterns in the data, and then making the MLP concentrate on the most essential information for final classification by making its dimension small.
- **MLP-Focused Approach (MLP Dimension: 128, FF Dimension: 64):** The aim of this combination is the opposite of the last one, by keeping the transformer FF focused on extracting general patterns, while giving more capacity to the MLP to model more complex class boundaries based on the features produced by the transformer.
- **Higher Capacity (MLP Dimension: 256, FF Dimension: 256):** To make the model capable of capturing very complex patterns, we made in this setup both the transformer FF and the MLP head large. This choice can increase parameter count, and improve accuracy if there is enough data.
- **Deeper MLP Network (MLP Dimension: [128,64], FF Dimension: 64):** The multiple MLP layers in this setup was used to progressively narrow the dimensionality (128→64). The two layers can learn deeper patterns. The FF dimension was set to match the MLP second layer.

The results of the experiments can be observed in Table 4.6 below.

Table 4.6: MLP and FF Dimension Tuning Experiments Result

Metric (MLP Dim., FF Dim.)	(64, 128)	(128, 64)	(256, 256)	([128, 64], 64)
Avg. Train Acc. (%)	97.58	98.02	98.44	95.68
Avg. Train Loss	0.0817	0.0694	0.0584	0.1274
Avg. Test Acc. (%)	82.21	82.49	82.70	81.73
Avg. Test Loss	0.8657	0.8553	0.8378	0.8126
Avg. MCC	0.8095	0.8126	0.8147	0.8046

the higher capacity (256, 256) achieved the highest average test accuracy (82.70 %) and MCC (0.8147), indicating that the extra capacity benefits both representation and decision boundaries at the expense of more parameters and greater computational cost. The MLP-Focused (128, 64) and Balanced (64, 128) configurations performed nearly, with test accuracy and MCC close to those of the (256, 256) configuration, although exhibited higher test loss. The Deeper MLP ([128,64], 64) surprisingly recorded the lowest test loss (0.8126) despite having the poorest test accuracy and MCC.

Depending on these results, **we will adopt a size of 256 for both the MLP Dimension and FF Dimension for the next experiments.**

Having established the optimal network capacity within individual transformer blocks, we now investigate how many encoder layers are needed to achieve sufficient model depth for complex pattern learning.

Number of Encoder Blocks: In this experiment, we will try to increase the number of encoder blocks, in order to reach a better performance for our model. The high memory consumption of the model prevented testing deeper encoder configurations, leading to test only two and three block encoders. Note that in the previous experiments, initial number of encoders was one block.

The results of the experiments can be seen in Table 4.7.

Table 4.7: Encoder Blocks Tuning Experiments Result

Metric (No. of Encoders)	2	3
Avg. Train Acc. (%)	94.40	93.69
Avg. Train Loss	0.1645	0.1821
Avg. Test Acc. (%)	82.22	81.77
Avg. Test Loss	0.8562	0.8637
Avg. MCC	0.8098	0.8050

Based on the experiments, the initial single encoder block is the best performing configuration (average test accuracy of 82.70% with the 256 MLP Dim. and 256 FF Dim. configuration see Table 4.6), while being more computationally efficient due to lower memory requirements. We can observe that increasing the number of encoders has only led to a decrease in the average test accuracy (from 82.70% to 82.22 with two blocks, and to 81.77% with three blocks). Thus, **we will continue using the initial single encoder block.**

With the optimal model depth now determined, the next step is to explore regularization techniques to prevent overfitting and ensure robust generalization across subjects.

Dropout and Regularization: In order to prevent overfitting and improve generalization, we will be testing different dropout rates applied to various model components along with bias regularization strength of the Conv1D layer of the Convolutional Feature Extractor (refer to Figure 3.8 in chapter 3).

- **Strong Regularization(Transformer Dropout: 0.4, MLP Dropout: 0.5, Conv1D Dropout: 0.4, Bias Reg.: 0.001):** In this combination we applied a strong dropout in attention layers to prevent overfitting in feature relationships, with a higher in MLP in the final layers where overfitting typically manifests most. The Conv1D Dropout was also increased from 0.3 in our current setup to 0.4 to reduce early-stage feature memorization. The bias regularization was also changed from 0.01 to 0.001 compared to the current setup combination.
- **Powerful Regularization(Transformer Dropout: 0.5, MLP Dropout: 0.6, Conv1D Dropout: 0.5, Bias Reg.: 0.005):** This setup is more stronger then the last one, with a high dropout in the transformer encoder, an aggressive dropout in classification layers and a strong early-stage regularization. To heavily penalize learned offsets a strong bias regularization is also applied.
- **More Powerful Regularization(Transformer Dropout: 0.55, MLP Dropout 0.65, Conv1D Dropout: 0.55, Bias Reg.: 0.005):** This combination is basically a more powerful one then the last setup and keeping the same bias regularization.

The results of the experiments can be observed in Table 4.8 below.

Table 4.8: Dropout and Bias Regularization Tuning Experiments Result

Metric (Transformer Drop., MLP Drop., Conv1D Drop., Bias)	(0.4, 0.5, 0.4, 0.001)	(0.5, 0.6, 0.5, 0.005)	(0.55, 0.65, 0.55, 0.005)
Avg. Train Acc. (%)	98.00	95.74	94.45
Avg. Train Loss	0.0662	0.1347	0.1689
Avg. Test Acc. (%)	82.67	82.98	82.21
Avg. Test Loss	0.9304	0.8761	0.9094
Avg. MCC	0.8145	0.8178	0.8096

Insufficient dropout and regularization retains overfitting risk, while too much underfits and degrades accuracy as the experiments demonstrates. Under the strong regularization (0.4, 0.5, 0.4, 0.001) configuration, the training accuracy is very high (98.00 %) compared to the test accuracy (82.67 %), which indicates a strong overfitting. The train accuracy have noticeably reduced in the Powerful Regularization (0.5, 0.6, 0.5, 0.005) configuration, with train accuracy of 95.74 % and the best test accuracy (82.98 %) and highest MCC (0.8178) indicating that the model is no longer memorizing so aggressively. The More Powerful Regularization (0.55, 0.65, 0.55, 0.005) configuration demonstrated that increasing the dropout a little more led to a slight decrease in the train accuracy (94.45 %) accompanied by a similar decrease in the test accuracy (82.21 %) and an increase in the test loss (0.9094).

Based on the experiments conducted, **the optimal dropout settings are identified as: transformer dropout, MLP dropout, Conv1D dropout, and bias reg of 0.5, 0.6, 0.5, 0.005 respectively.**

With all hyperparameters systematically optimized, we have achieved our final model configuration that balances performance, generalization, and computational efficiency.

Summary of Hyperparameter Tuning

With the completion of the dropout and regularization experiments, our systematic hyperparameter tuning process is finalized. Under the chosen hyperparameter configurations, the model has achieved an **average training accuracy of 95.74 %**, an **average train loss of 0.1347**, an **average test accuracy of 82.98 %**, an **average test loss of 0.8761**, and an **average Matthews Correlation Coefficient of 0.8178**. Figure 4.2 shows how the test accuracy improved through each phase of optimization, the encoder block phase is not included in the graph since no improvement was made in that experiment.

We also present the global confusion matrix of this final model in Figure 4.3, and the detailed performance results for each of the 40 subjects in Table 4.9.

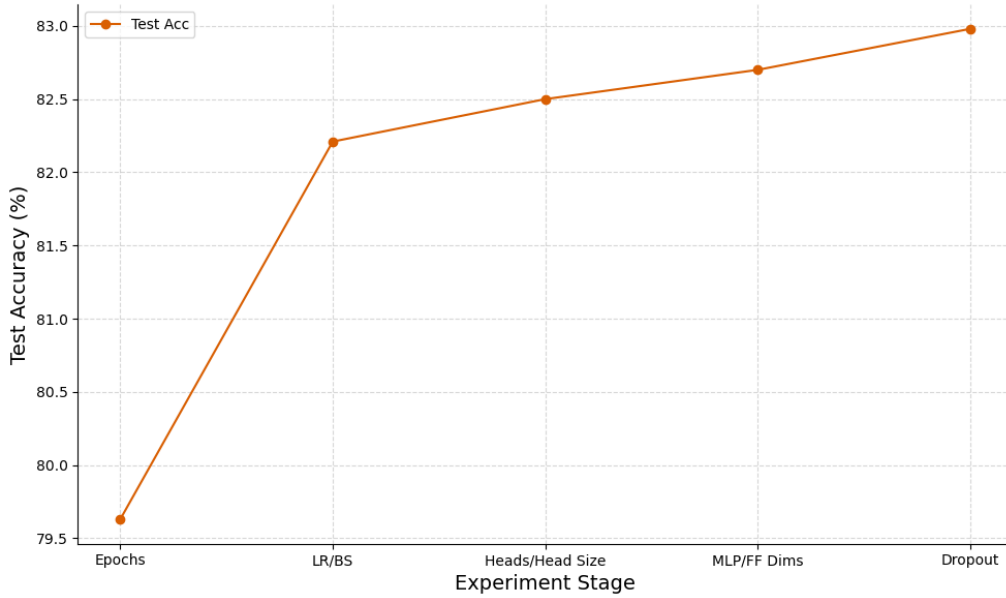


Figure 4.2: Model Test Accuracy Improvement Through Optimization

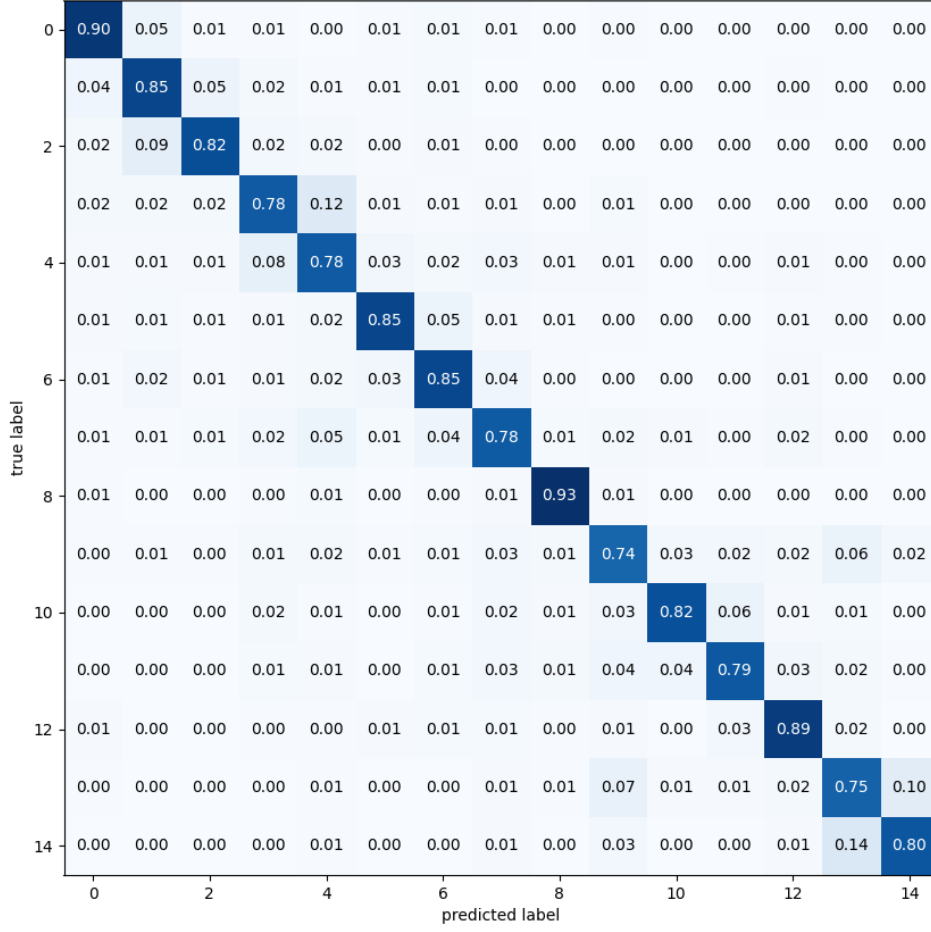


Figure 4.3: Global Confusion Matrix of Final Model

The presented matrix is a row-normalized 15x15, where each cell shows the proportion of true label examples (rows) predicted as each class (columns).

It can be seen that many diagonal entries are ≥ 0.80 (e.g. class 8 with 0.93, class 12 with 0.89), indicating that the model correctly recognizes these gestures over most of the time, although there is some gestures that the model struggles with, such as class 9 that has the lowest recall at 0.74, and class 13 with 0.75. A notable confusions between specific classes can be observed:

- **Class 3 vs. 4:** Class 3 is misclassified as class 4 about 12 % of the time, and 8 % of true class 4 samples were wrongly predicted as class 3.
- **Class 2 vs. 1:** Class 2 is misclassified as class 1 around 9 % of the time.
- **Class 14 vs. 13:** Class 14 is wrongly predicted as 13 about 14 % of the time (and 0.10 in the reverse direction).

The confusions between those pairs suggest that they have similar EMG patterns, leading the model to get confused between them.

Table 4.9: Performance Metrics for Subjects 1–40

Subject	Train Acc. (%)	Train loss	Test Acc (%)	Test loss	MCC
Subject 1	96.88	0.0817	89.30	0.5608	0.8851
Subject 2	98.51	0.0625	87.55	0.7177	0.8666
Subject 3	98.27	0.0794	87.19	0.7410	0.8625
Subject 4	95.72	0.1398	79.05	1.1284	0.7756
Subject 5	93.63	0.1808	86.16	0.5277	0.8501
Subject 6	98.04	0.0895	83.86	0.8273	0.8262
Subject 7	93.96	0.2218	73.10	1.2241	0.7126
Subject 8	99.73	0.0246	88.08	0.8169	0.8730
Subject 9	98.50	0.0621	83.77	0.9851	0.8264
Subject 10	98.61	0.0488	88.51	0.6601	0.8766
Subject 11	91.37	0.2755	78.93	0.9049	0.7749
Subject 12	94.68	0.1495	84.72	0.8555	0.8359
Subject 13	98.73	0.0555	86.32	0.8250	0.8540
Subject 14	87.94	0.3321	74.95	0.7854	0.7321
Subject 15	96.92	0.0970	85.24	0.7441	0.8410
Subject 16	97.62	0.0978	75.91	1.1867	0.7439
Subject 17	97.52	0.0844	88.06	0.6037	0.8718
Subject 18	92.41	0.2234	82.24	0.7611	0.8091
Subject 19	97.06	0.1093	81.49	0.8326	0.8023
Subject 20	96.86	0.0987	84.09	0.7612	0.8293
Subject 21	94.79	0.1577	78.11	1.1236	0.7646
Subject 22	99.17	0.0413	86.07	1.0380	0.8514
Subject 23	98.00	0.0801	81.96	1.0055	0.8070
Subject 24	92.26	0.2254	72.52	1.6944	0.7061
Subject 25	98.03	0.0632	91.43	0.4744	0.9084
Subject 26	93.19	0.1893	82.39	1.0947	0.8127
Subject 27	96.45	0.1091	83.96	0.7916	0.8281
Subject 28	92.80	0.2570	74.14	1.1661	0.7232
Subject 29	97.84	0.0629	83.04	1.2563	0.8182
Subject 30	95.70	0.1242	80.05	0.6828	0.7865
Subject 31	79.50	0.5849	73.24	1.1051	0.7142
Subject 32	95.39	0.1368	84.28	0.7918	0.8326
Subject 33	97.79	0.0813	93.73	0.3271	0.9327
Subject 34	98.39	0.0530	82.12	1.1994	0.8101
Subject 35	94.94	0.1723	79.45	1.0343	0.7795
Subject 36	98.20	0.0735	81.75	1.0982	0.8046
Subject 37	97.33	0.0785	89.32	0.5104	0.8852
Subject 38	92.37	0.2076	82.69	0.7029	0.8153
Subject 39	96.00	0.1333	82.37	0.9066	0.8112
Subject 40	98.68	0.0436	88.21	0.5932	0.8735

The results show a variability in performance across subjects, with the test accuracy spanning a wide range. Subjects like 7, 14, 24, 28 and 31 show notably lower test accuracies and MCCs compared to the top performers. It is noticeable that for Subject 31, even the training accuracy (79.50 %) is low, suggesting the model struggled to even learn its training data, making it an exception compared to others. For most of the other low performing subjects, the training accuracy is high, emphasizing the generalization challenge.

Training accuracy for most of the subject is high compared to the test accuracy, indicating a degree of overfitting. The Matthews Correlation Coefficient (MCC) values largely mirror the test accuracy results, providing a balanced measure of classification quality. To better visualize the performance on each subject we present Figure 4.4 below:

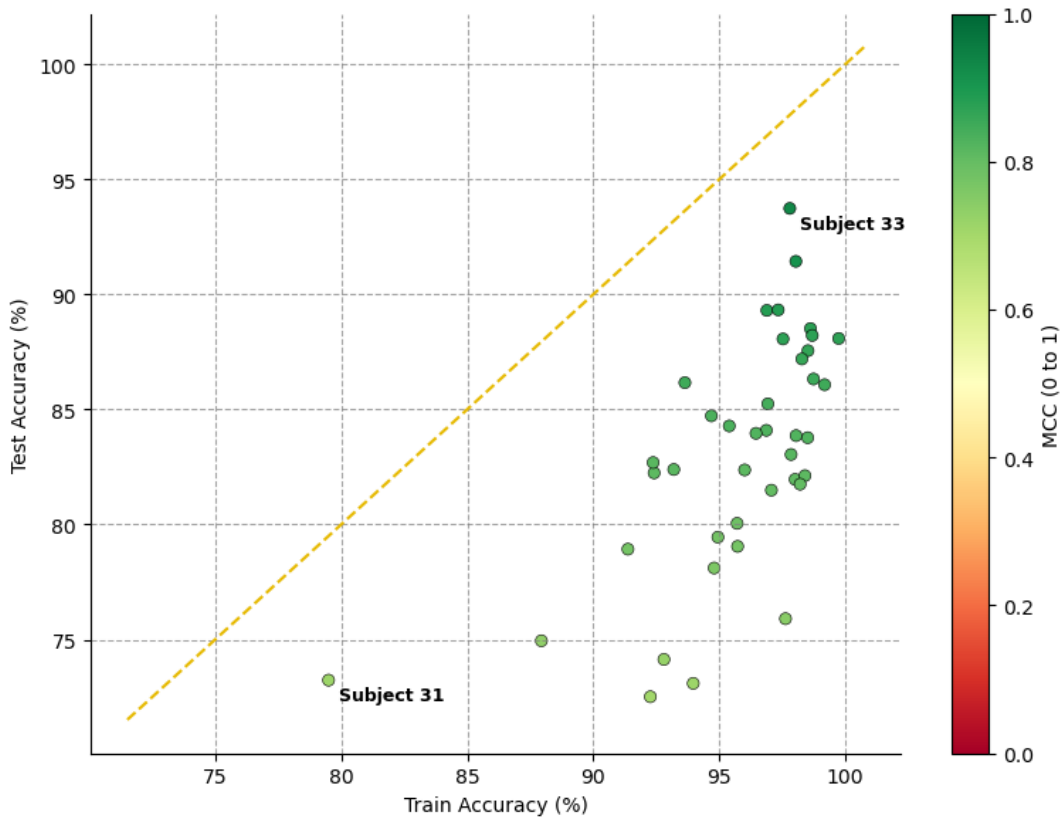


Figure 4.4: Train vs Test Accuracy (Color = MCC)

The X-axis represents the train accuracy (values increase from left to right), and the Y-axis represents test accuracy (values increase from bottom to top). MCC is represented by a gradient color scale starting from 0 (prediction no better than random) to +1 (perfect prediction), and the diagonal line (train accuracy = test accuracy) highlights the overfitting subjects, the points that are extremely below the line indicate subjects where the model overfit.

The majority of subjects cluster in the 95-100 % train accuracy range with test accuracies between 80-95 %. It can be seen that some subjects with very high train accuracy (> 98%) have a moderate test accuracy (80-90 %), demonstrating that the

model is overfitting on some subjects as discussed before. The balanced classification performance is plainly visible, since the color gradient shows that higher MCC values (greener points) tend to cluster in the upper portion of the plot (correlates with higher test accuracy).

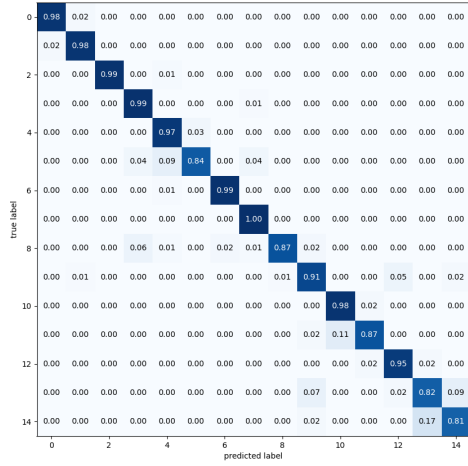
To get a better close look at the the low and high performing subjects, we present Table 4.10 which contains results for the best and worst case subjects.

Table 4.10: Results for Best (S33) and Worst (S24) Subjects for the Final Model Configuration

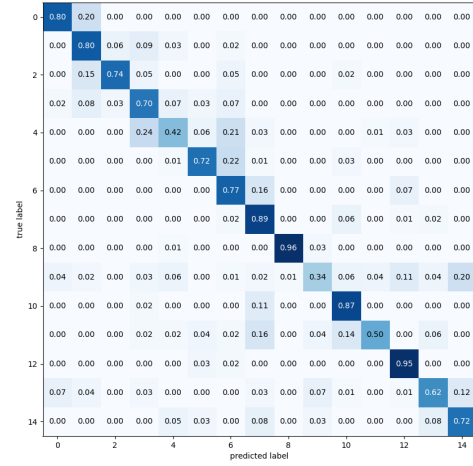
Subjects	Subject 33	Subject 24
Train Acc. (%)	97.79	92.26
Train Loss	0.0812	0.2253
Test Acc. (%)	93.73	72.52
Test Loss	0.3270	1.6944
MCC	0.9326	0.7060

Subject 33 which is the best case achieved excellent test accuracy of 93.73 % and an high MCC of 0.9326 demonstrating an almost perfect predictions. This results shows that the model is performing equally well across all classes, this can be seen in the confusion matrix of subject 33 in Figure 4.5. The gap between the test and train accuracy is small, indicating a good generalization for this subject.

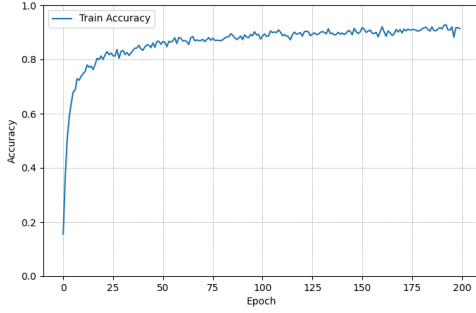
Subject 24 which is the worst case, shows a much lower test accuracy of 72.52 % and an MCC of 0.7060. For the train accuracy, it is much larger then the test accuracy (92.26 %), highlighting a bad generalization of the model for this subject.



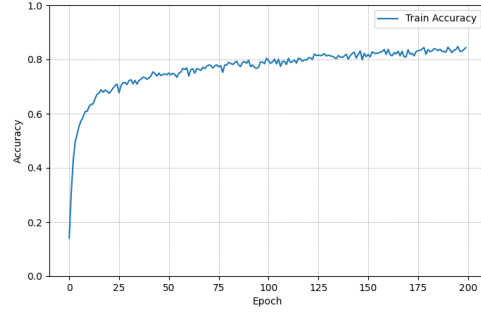
(a) Confusion Matrix — Subject 33



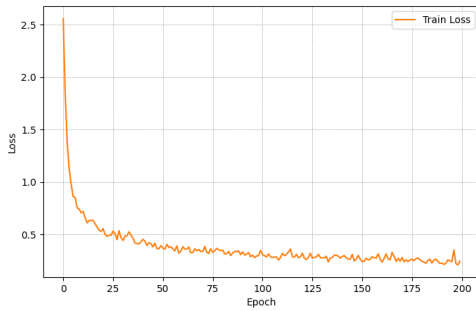
(b) Confusion Matrix — Subject 24



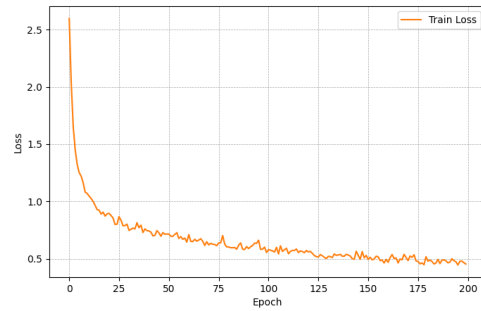
(c) Train Accuracy — Subject 33



(d) Train Accuracy — Subject 24



(e) Train Loss — Subject 33



(f) Train Loss — Subject 24

Figure 4.5: Confusion Matrices and Training Curves for Best (S33) and Worst (S24) Subjects Using the Final Model Configuration.

After successfully tuning our model, we need to check if the pre-processing techniques we applied to our signals have an impact on those final results. Therefore, the next section will discuss their effect on the model performance.

4.5.2 Effect of the Pre-processing Techniques on the Model Performance

In order to demonstrate the effect of the pre-processing techniques applied to the data before feeding it to the model, we are going to see how the model will perform on the raw data.

Table 4.11: Model Results on Raw Data

Metric	Raw Data	Model Final Results(Pre-processed)
Avg. Train Acc. (%)	12.91	95.74
Avg. Train Loss	2.8586	0.1347
Avg. Test Acc. (%)	12.26	82.98
Avg. Test Loss	2.9231	0.8761
Avg. MCC	0.0504	0.8178

The results of the model performance on the raw data, shown in Table 4.11, provides a dramatic illustration of how crucial pre-processing is for our model success:

- Test Accuracy: 82.98% (preprocessed) vs. 12.26% (raw), indicating a 70.72% improvement.
- Training Accuracy: 95.74% (preprocessed) vs. 12.91% (raw), indicating a 82.83% improvement.
- MCC: 0.8178 (preprocessed) vs. 0.0504 (raw), indicating a 0.7674 improvement.

These improvements can be clearly seen in Figure 4.6 below.

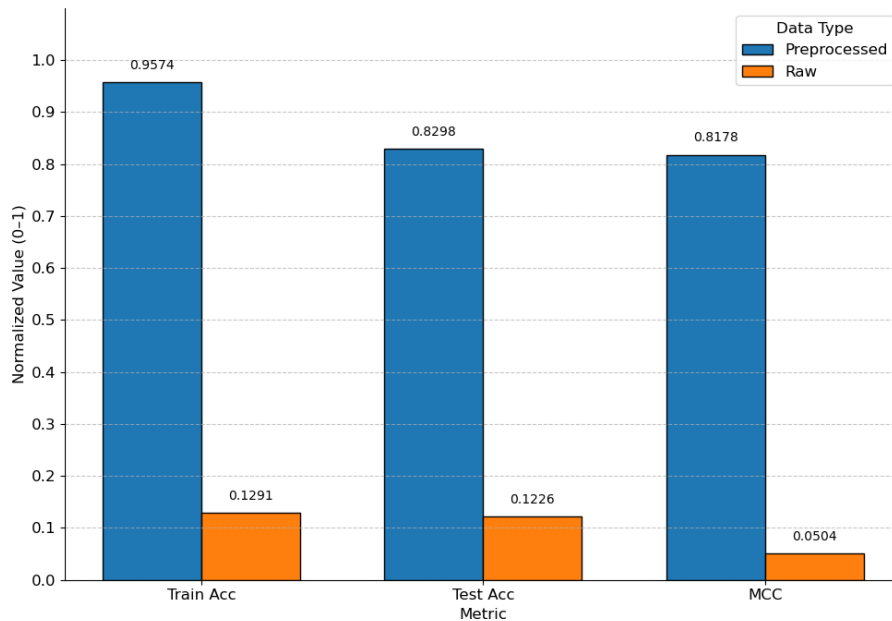


Figure 4.6: Comparison of the Model Performance on Raw and Preprocessed Data

Random guessing on a 15-class task would likely yield a 6.67 % test accuracy. This is slightly below the raw data results, which is 12.26 %. This could indicate that the model cannot extract meaningful patterns from the unprocessed signals since it may contain too much noise and artifacts and the signal-to-noise ratio is too poor for direct learning. The model classifications are mostly random, and this can also be proven by the MCC value which is almost 0 (0.0504).

High loss values for both training (2.85) and test (2.92), suggest that the model is struggling to converge during training and the predictions are highly uncertain and scattered across classes.

It is also notable that the train and test accuracy (12.91% and 12.26%) are almost identical, demonstrating that the model is not even overfitting, but it simply cannot learn anything meaningful.

Having established the pre-processing impact, we will analyze the model computational efficiency in the next section.

4.5.3 Time Complexity Analysis

After highlighting the pre-processing effect on the model performance, we will now analyze the computational efficiency, which is an important factor in real-time EMG gesture recognition systems. The analysis includes measuring the average training and testing time per window across the 40 subjects in our data set. For each subject, we divide their total training and testing time by the number of windows in their train and test sets to get per-window times, the resulting per-window test and train times are then averaged across all subjects.

We note that the test/train time of each subject were taken during our final model experiment that achieved test accuracy of 82.98%, which was recorded on a Google Colab T4 GPU. It is also important to note that the raw data was processed using a 150 ms window (300 samples) and a 100 ms stride (200 samples) as was previously mentioned in chapter 3.

Table 4.12 summarizes the train and test time performance across subjects, including ranges of the time per-window to assess variability across subjects.

Table 4.12: Training and Testing Time Metrics Across 40 Subjects

Metric	Training Time	Testing Time
Total Time Across Subjects (min)	67.63	1.31
Average Total Time per Subject (s)	101.45	1.96
Average Time per Window (s)	0.046484	0.001862
Per-Window Time Range (s)	0.033262–0.068706	0.000191–0.003369

The total training time across all subjects (67.63 min) seems reasonable for training across a diverse dataset. On the other hand, the total test time is small (1.31 min) indicating that the model is fast at making predictions, which is valuable for real-time applications. Testing a single subject on average will take 1.96 seconds, which supports practical deployment, while training one subject will take on average 101.45 seconds.

The average test time per window is 0.001862 seconds (1.86 ms), which is significantly less than the 150 ms window duration, this enable the model to process the window much ahead of new window, since in a real-time scenario with a 100 ms stride, there are 100 ms before the next window arrives, so the model can process windows a lot faster than they are generated. This is more than sufficient for real-time application. This can be shown clearly in Figure 4.7 where the safety margin (the leftover time between when the model finishes processing a window and when the next window arrives) is large.

The average train time per window is 0.046484 seconds (46.48 ms), reflecting the higher computational cost of training over 200 epochs. Since training can be done offline the 100 ms interval does not create a real-time constraint.

The range for both training and testing time per window indicates the variability in times across subjects. Outliers include Subject 28 with a test time of 0.000191 seconds and Subject 40 with a train time of 0.068706 seconds (likely due to noisy data).

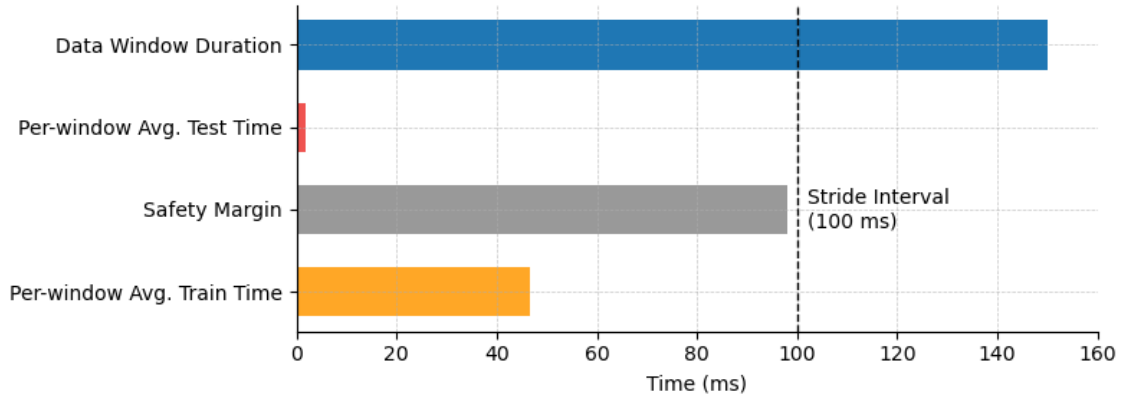


Figure 4.7: Real-Time Feasibility Timeline

After we have seen the time complexity of our model, we move now to the comparison part to check if the model has competitive performance against other models that were tested on the same database.

4.5.4 Performance Comparison with Other Models

To evaluate the competitiveness of our proposed model, we will compare its performance against three other models: CNN-1D, CNN-2D, and CNN-LSTM. The models were tested on the same DB2, on all the 40 subjects and the same 15 hand gestures. Since the window size, the stride, and the hardware configurations used on these models are different from ours, the time complexity comparisons will be unreliable and are therefore not included. In our comparison, we will focus on the test accuracy and Matthews Correlation Coefficient (MCC), and model complexity as indicated by the total and trainable parameters. Table 4.13 summarizes the performance and complexity of all models.

Table 4.13: Performance and Complexity Comparison of the Models

Metric	Our Proposed Model	CNN-1D	CNN-2D	CNN-LSTM
Avg. Test Accuracy (%)	82.98 $\pm$ 5.34	75.35 $\pm$ 5.93	73.12 $\pm$ 6.48	73.79 $\pm$ 6.19
Avg. MCC	0.8178	0.7379	0.8009	0.7216
Total Parameters	128,536	47,536	342,256	444,176
Trainable Parameters	128,408	47,408	342,256	443,664

Our proposed model has achieved both the highest average test accuracy (82.98%) and MCC (0.8178), outperforming all the other three models. The CNN-1D model ranked second, with average test accuracy of 75.35% and MCC of 0.7379, showing a gap of 7.36% and 0.0799 respectively. It is followed closely by the CNN-LSTM model, with average test accuracy of 73.79% and an MCC of 0.7216, and by CNN-2D model, with average test accuracy of 73.12%, MCC of 0.8178. Our model achieved the lowest test accuracy standard deviation (5.34) among the models, showing its relative stability across subjects. Based on that, the accuracies will typically vary between 77.64% (82.98 - 5.34) and 88.32% (82.98 + 5.34), with extremes at 72.52% and 93.73%. Figure 4.8 demonstrates the highest and lowest test accuracies for the four models.

Regarding model complexity, our proposed model demonstrates an optimal balance between performance and parameter efficiency. With 128,536 total parameters (128,408 trainable), it significantly outperforms the more compact CNN-1D model (47,536 total parameters) while maintaining superior computational efficiency compared to the CNN-2D model (342,256 parameters) and especially the CNN-LSTM model (444,176 parameters). This efficiency advantage is particularly notable when considering that our model achieves the highest accuracy with fewer parameters than both CNN-2D and CNN-LSTM, demonstrating the effectiveness of the transformer architecture in learning complex temporal patterns without excessive parameterization.

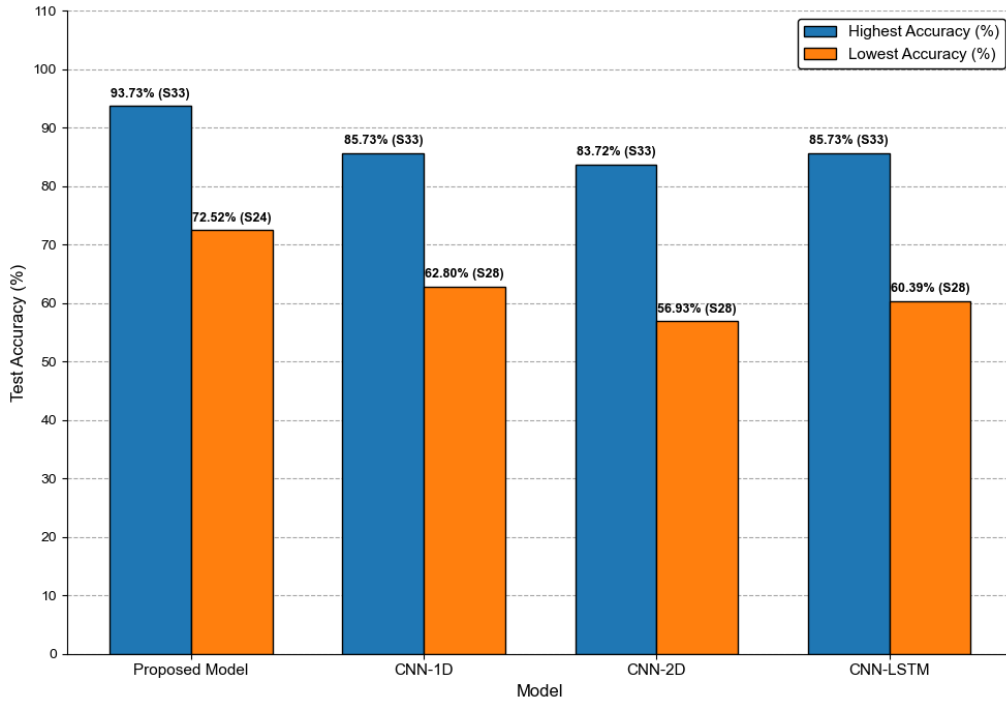


Figure 4.8: Highest and Lowest Test Accuracies Across Models with Corresponding Subjects.

The four models aligned with the proposed model in identifying subject 33 as favorable, likely due to Subject 33’s EMG signals exhibiting minimal noise and highly consistent gesture execution patterns. However, the recurring struggle with Subject 28 for CNN-1D, CNN-2D, and CNN-LSTM, and Subject 24 for the proposed model, still points out to the challenge in handling inter-subject variability.

In order to compare the subject-wise performance, we present Figure 4.9, which shows the test accuracy across all 40 subjects for both our proposed model and CNN-1D (the best performing model among comparison ones).

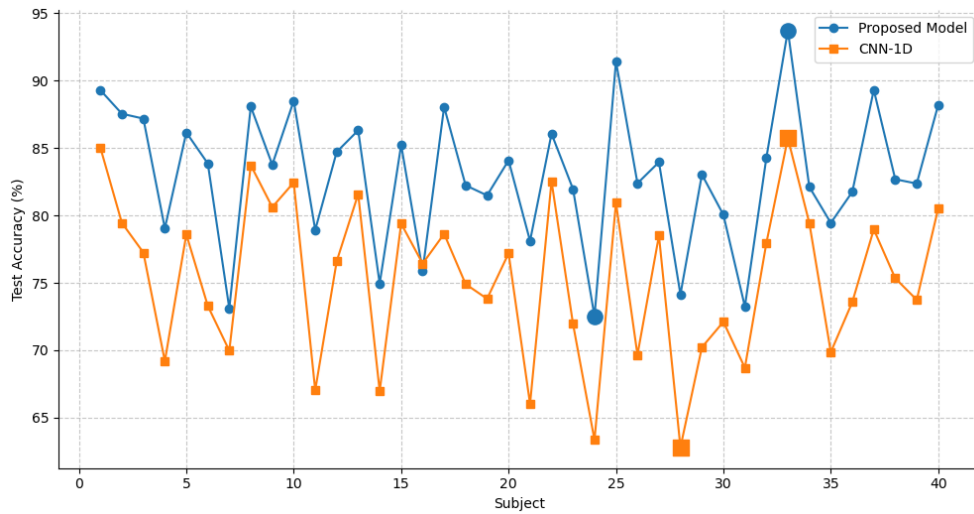


Figure 4.9: Test Accuracy Across All Subjects: Proposed Model vs. CNN-1D.

The proposed model accuracy curve (represented by the blue line) consistently tracks above the CNN-1D model (orange line), reflecting its superior average performance. We can observe that both models perform well on subjects with cleaner signals (e.g., 1–3, 8–10, 33), but the proposed model never drops below 72.52 %, highlighting its robustness.

The remarkable performance by our proposed transformer-based model can be attributed to a number of factors. The transformer architecture, especially the attention mechanism, which was very helpful in distinguishing the 15 gestures, and helped in improving the performance stability across subjects as can be seen in Figure 4.9, and by the low standard deviation compared to other models. Additionally, our pre-processing pipeline that ensured cleaner input signals and improved the average test accuracy by 70.72% over raw data (section 4.6), and likely mitigated some of the inter-subject variability, as evidenced by its higher minimum accuracy compared to other models.

The three comparison models are maybe effective in feature extraction, but they may not capture the same level of temporal complexity as our transformer-based model. Although the CNN-2D model achieved an average MCC of 0.8009, it still falls short of our model’s overall performance.

4.6 Conclusion

In this chapter, the software environment, model implementation, and experimental protocols used to develop and evaluate our model have been presented. A robust average test accuracy of 82.98% and an MCC of 0.8178 was achieved through the hyperparameter tuning experiments, and the results highlighted the critical role of pre-processing pipeline that we adapted, which boosted the performance from near-random classification on raw data to high accuracy and stability on clean signals. Our time complexity analysis demonstrated real-time feasibility with an average testing time of 1.86 ms per window. Furthermore, the comparison made against CNN-1D, CNN-2D, and CNN-LSTM models on the same DB2 dataset confirmed the superiority of our approach, with our model achieving the highest accuracy and lowest performance variability across subjects. Additionally, our model demonstrates exceptional parameter efficiency, achieving superior performance with moderate complexity (128,536 parameters) compared to the over-parameterized CNN-LSTM (444,176 parameters) and CNN-2D (342,256 parameters) models, highlighting the effectiveness of our transformer-based architecture in balancing model capacity with computational efficiency. This results demonstrate how effective our model is, for robust EMG-based gesture recognition systems.

Conclusion

Automatic hand gesture recognition is an appealing research and engendering field with a wide range of practical applications. Our project has explored the transformative potential of Transformer-based models for hand gesture recognition using electromyographic (EMG) signals, advancing the field of human-machine interaction through innovative signal processing techniques. Through meticulous experimentation and comparative analysis, we have demonstrated the remarkable capabilities of leveraging Transformer architecture for the task under hand. Our findings firmly establish that the Transformer approach excels in classifying hand gestures from EMG data, particularly when enhanced with tailored pre-processing and architectural refinements. Our proposed model consistently outperforms traditional deep learning models included in the comparison, showcasing superior efficiency in capturing temporal dependencies and global contextual information. This reaffirms the Transformer effectiveness as an optimal choice for precise and reliable gesture recognition in our research.

Our study underscores the importance of advancing AI integration in human-machine interfaces, with significant implications for applications such as prosthetics, virtual reality, and assistive technologies. By enabling more accurate and responsive gesture recognition.

The primary contribution of this research lies in the successful application and validation of a Transformer-based model for multi-class hand gesture recognition from EMG signals, demonstrating substantial robustness across subjects and real-time inference capability. Additionally, contributing to the development of smarter, more intuitive systems that enhance user experiences and accessibility, particularly for individuals with motor impairments.

Future efforts will focus on enhancing the model scalability and exploring hybrid techniques to push performance boundaries further, and address the encountered limitations by trying to further improve generalization across subjects.

In conclusion, this project demonstrates that Transformer-based models offer a promising and powerful approach for hand gesture recognition from EMG signals. The ability of these models to capture contextual information within EMG time-series data holds significant potential for the development of more robust, accurate, and intuitive myoelectric control systems. Further refinement and exploration in this direction are warranted to fully harness the capabilities of Transformers in this important field.

Appendix A Main Deep Learning Architectures

This appendix provides foundational knowledge of the main deep learning architectures used in time series classification tasks, particularly for EMG signal processing. Additionally, it outlines the core training parameters referenced in our experiments.

A.1 CNN-based Methods

Convolutional neural networks (CNNs) exhibit superior performance compared to other neural network architectures, particularly in tasks involving image, speech, or audio signal processing. As described in [49], CNNs consist of three main types of layers:

- Convolution layer
- Pooling layer
- Fully-connected (FC) layer

The convolutional layer typically initiates the CNN architecture. Convolution and pooling layers are typically stacked, with a fully connected layer used as the final output stage. The complexity of features extracted by the CNN increases with each successive layer, progressing from simple patterns such as colors and edges to higher-level features, including object shapes and structures, ultimately enabling object recognition [49].

Convolution Layer:

The convolutional layer serves as the core building block of a CNN. Figure A.1 shows the idea behind the convolution, also known as cross-correlation. This process operates as a sliding window over a grid of data, where a filter moves across the grid, multiplying the filter's weights with the corresponding parts of the data and adding them up as it goes. This helps to extract useful information from the data grid. By learning the appropriate filter weights during training, CNNs can automatically extract meaningful features from the input data, enabling them to perform tasks like image classification [51].

Figure A.1 illustrates the process where a 3x3 kernel is slid across the input signal, which is initially of size 4x4. With each stride, the kernel traverses the input,

performing the convolution operation. As a result, we obtain a feature map reduced in size from 4x4 to 2x2.

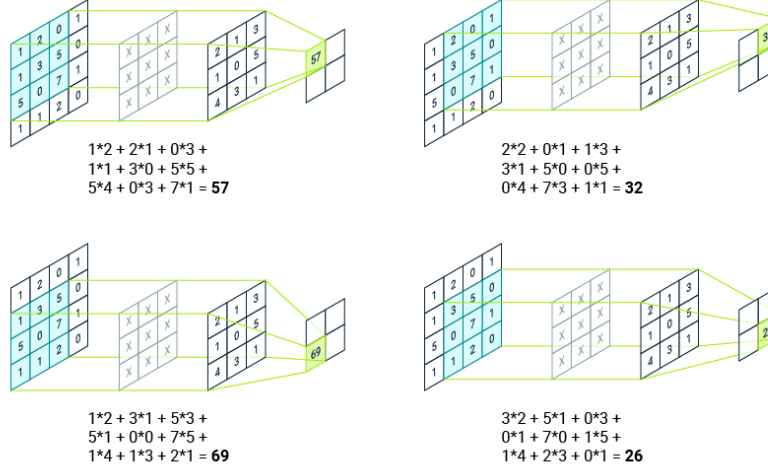


Figure A.1: Single Channel Convolution Visualization [50].

However, in a CNN, the input signal may consist of multiple channels as well. When a signal represents an image, each channel is a pixel matrix of a base color. Consequently, the convolution kernel applied in such cases must also be multi-channel, with each channel having its own weight matrix. Figure A.2 illustrates how the convolution process is carried out in such a scenario for a single kernel position (single stride). For each channel of the input signal, the corresponding channel of the kernel is subjected to element-wise multiplication and summation. The results for each channel are then aggregated. Consequently, a single reduced matrix is obtained. A typical example is a spectrogram image represented in 4-channel RGBA format [51].

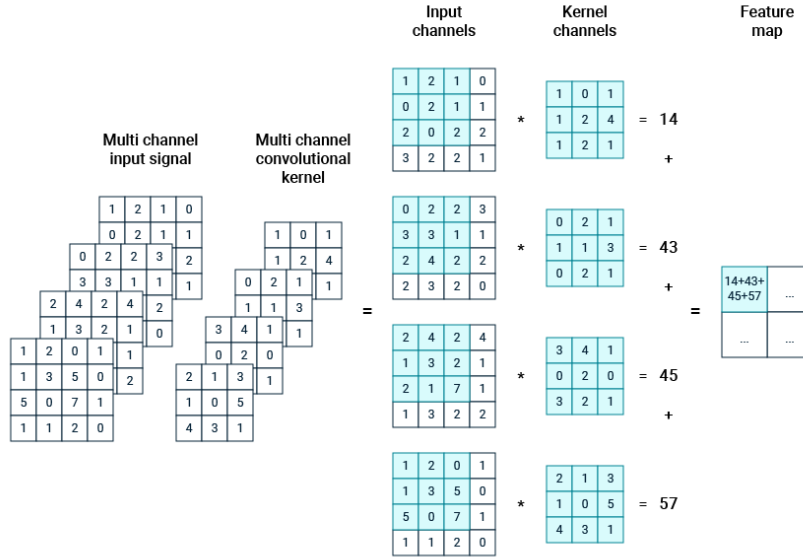


Figure A.2: Multi-channel Convolution Visualization[50]

Pooling Layer:

After the convolutional layers, the pooling layers perform down-sampling of the feature maps, effectively reducing their spatial dimensions while retaining essential information. In Figure A.3, the pooling layer with a kernel size of 2x2 is depicted performing a maximum operation on the input layer, which condenses the output matrix dimension to 2x2. Using a stride of two, the kernel shifts both horizontally and vertically across the input. In pooling layers, aside from the max operation, other aggregating operations like average pooling, sum pooling, and several others can be used [51].



Figure A.3: Example of Pooling Layer Visualization [50].

Fully Connected Layers:

Fully connected layers, also referred to as dense layers, establish connections between every neuron in one layer and every neuron in the subsequent layer. These layers play an important role in classification tasks by combining features learned from preceding layers, typically flattened into a one-dimensional input vector, to be associated with the output classes. Although pixel values can constitute the input

vector, they more commonly encompass flattened feature maps derived from the image via convolutional layers [51].

Dropout:

Another typical characteristic of CNNs is the Dropout layer. The dropout layer is a mask that nullifies the contribution of some neurons towards the next layer and leaves all the others unmodified [52]. Figure A.4 shows the effect of the dropout on the neurons.

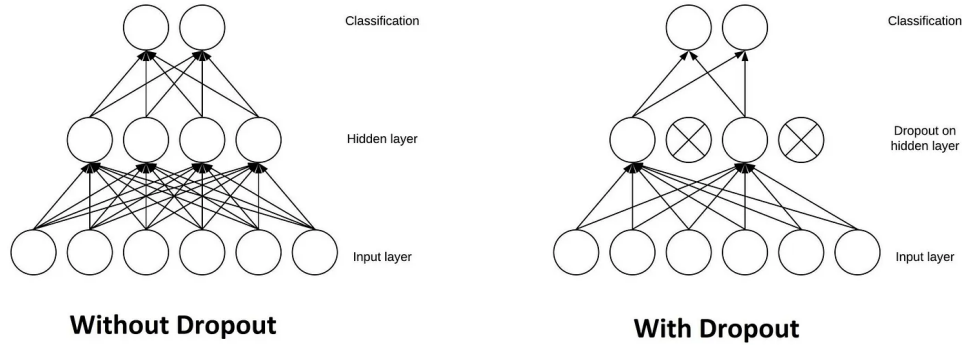


Figure A.4: With and Without Dropout[52]

Activation Function:

An activation function decides whether a neuron should be activated. This means that it will decide whether the input of the neuron to the network is important or not in the prediction process. There are several commonly used activation functions such as the ReLU, Softmax, tanH, and the Sigmoid functions. Each of these functions has a specific usage.[53]

A.2 RNN-based Methods

Recurrent Neural Networks (RNNs) were developed to overcome the limitations of traditional neural networks, such as Feedforward Neural Networks (FNNs), in processing sequential data. Unlike FNNs that process each input individually through multiple hidden layers with no regard to the order or context, RNNs deal with the problem of modeling input dependencies. This makes FNNs less ideal for applications requiring sequential relationship comprehension, such as language modeling, machine translation, speech recognition, and time series analysis.

To better illustrate the difference, Figure A.5 shows a Feedforward Neural Network on the left and a Recurrent Neural Network on the right. As depicted, the Feedforward Neural Network processes input through a series of layers without maintaining information between steps. In contrast, the Recurrent Neural Network introduces a feedback loop, allowing information from previous time steps to influence the current output. This recurrent connection, highlighted in Figure A.5b and absent in Figure A.5a, enables RNNs to form an internal memory and effectively model sequential data, as detailed by [54].

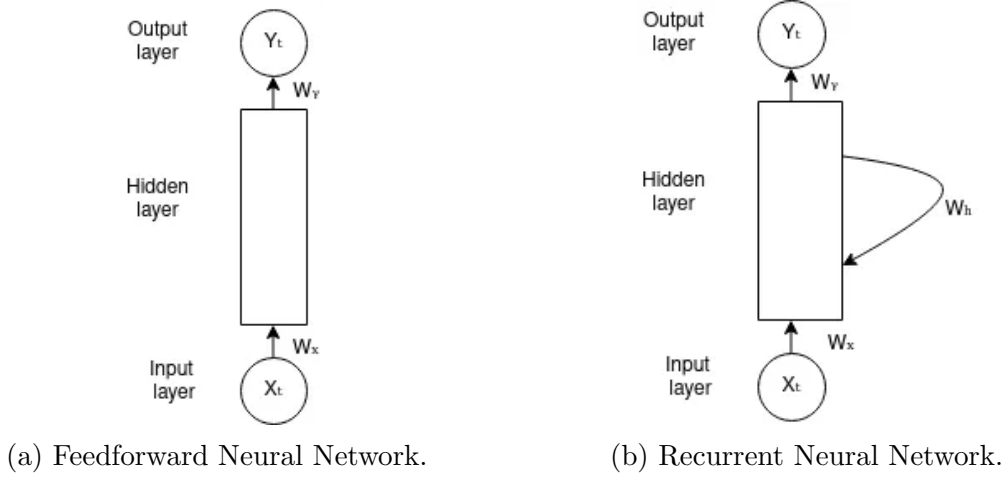


Figure A.5: Comparison of Feedforward and Recurrent Neural Networks [54].

RNNs overcome these constraints by introducing a recurrent connection where information can flow from one time step to another. This architecture allows RNNs to form an internal memory, where the output at each time step is recycled as input to the following step. By this, the network can utilize knowledge from earlier steps to learn time dependencies and process inputs of different lengths in an efficient way.

RNN Architecture

The architecture, illustrated in Figure A.6, accepts an input vector (X) and produces an output vector (y) handling data one step at a time, from left to right. As new data is received at each time step, the hidden state is updated, and an output is generated using the same set of parameters. The weight parameters (U), (V), and (W) consistently serve the same functions: (U) connects the input layer (X) to the hidden layer (h), (W) manages the connections between the hidden states across time, and (V) links the hidden state (h) to the output layer (y). By applying these shared parameters iteratively, as shown in the unfolded representation of the RNN in Figure A.6, the network effectively captures temporal dependencies within the data and processes sequential information by retaining knowledge of past inputs in its current hidden state, as explained by [54].

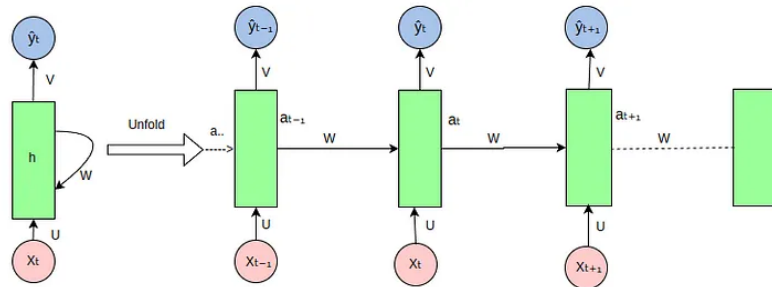


Figure A.6: RNN Unfolded[54]

Limitations of RNN

During backpropagation, which is the process of adjusting network weights based on errors, gradients propagated backwards in time can face the vanishing gradient problem. This occurs when gradient values become too small to make significant changes to the network weights, particularly in earlier layers [54]. It is a result of the successive multiplication of gradients through the chain rule, leading to an exponential decay in their magnitude, especially problematic for deep models or sequences with long-term dependencies. While models may eventually converge, these extremely small updates hinder the learning of long-range temporal trends.

On the other hand, the exploding gradient phenomenon occurs when gradients assume very large values, causing numerical instability during optimization. These large gradients can destabilize the training process by overshooting optimal solutions, making it difficult to converge to useful minima. This issue is particularly severe in recurrent architectures due to the compounding of gradients across time.

To counter these issues, methods such as gradient clipping, the use of architectural improvements like Long Short-Term Memory (LSTM) networks, and gradient normalization are commonly employed. These methods improve training stability by either controlling the size of the gradient or by restructuring the network to facilitate a smoother flow of information over long sequences [55].

A.2.1 CNNs and RNNs in Time Series Classification

The emergence of deep learning has largely revolutionized the field of time series classification largely due to advances in feature extraction techniques. For example, Convolutional Neural Networks (CNNs) have proven to be very effective in learning hierarchical representations of unstructured time series information using localized filters to identify patterns regardless of their locations within the sequence. This feature is particularly useful in applications like human activity recognition and anomaly detection, where localized temporal patterns must be detected to make accurate classifications [56]. Unlike CNNs, Recurrent Neural Networks (RNNs) and their variants such as Long Short-Term Memory (LSTM) and Echo State Networks are designed to learn sequential dependencies through information retention in their internal states, making them very well-suited to applications where event sequences play a significant role. Though cited in [56], RNNs with their effective capacity to incorporate temporal evolution might have difficulties in efficiency at training and handling long sequences compared to CNN-based designs. In summary, CNNs are used mostly to extract and classify intricate shift-invariant patterns in time series information while RNNs are used due to their capacity to maintain temporal context over successive observations [57].

A.2.2 CNNs and RNNs Limitations for Time Series Classification

Although both CNNs and RNNs have achieved success, they have significant issues classifying time series data. CNNs tend to require a great deal of labeled training data to perform well and are sensitive to design choices such as filter sizes,

stride values, and the number of layers. They also behave like "black boxes," so it is difficult to see what they learn as features, something particularly important in domains in which it is required to see the way that choices were made [59, 60]. RNNs suffer issues such as vanishing or exploding gradients, making it difficult to learn long-term connections. Their sequential processing of data also results in longer training times and lower speeds that is not good for very long sequences or real-time processing. Furthermore, as demonstrated in forecasting studies such as that found in [61], even sophisticated RNNs will struggle to maintain good performance over long durations. In summary, although both have advantages, we must carefully consider their shortcomings while developing models for particular time series classification tasks.

A.3 Training Parameters

For the sack of easier understanding of the experimental part, we remind the key involved concepts in our experiments.

- **Learning rate:** This controls how big of a step the model takes when tweaking its internal settings (weights) to improve predictions. A fixed rate is like a steady jog, but some optimizers switch it up dynamically to find the sweet spot faster.
- **Batch size:** This is how many data samples the model processes before updating its weights. Bigger batches can speed things up, especially on powerful hardware, but go too big, and the model might struggle to learn general patterns.
- **Epoch:** One complete run through the entire training dataset. Each epoch is like a full study session where the model reviews all the data, updating its weights after each batch.
- **Batch normalization:** A trick to keep the model's calculations stable. It's a layer that standardizes the data flowing through the network, added after convolutional or dense layers but before activation functions. It speeds up training and helps prevent overfitting.
- **Dropout:** A clever way to stop the model from getting too attached to specific neurons. During training, it randomly turns off a portion of the neurons, forcing the network to be more flexible and reducing the risk of overfitting.
- **Weight regularization:** This adds a penalty to the model's loss function to keep weights from getting too large. The popular ℓ_2 regularization nudges the model toward simpler, smaller weights, which helps it generalize better.
- **Loss function:** The scorecard for how well the model's predictions match the actual data. A lower score means the model's doing a great job, and the goal is to minimize this value during training.

Appendix B

B.1 Technologies

- **Python**¹: Open-source, high-level scripting language known for clean syntax, readability, and versatility. Widely used in data science, artificial intelligence, and machine learning, with a rich ecosystem of libraries (e.g., Scikit-learn, TensorFlow, PyTorch) that simplify complex computational tasks. Python 3.12 was used for this project.
- **Google Colaboratory (Colab)**²: Cloud-based Jupyter notebook environment for writing and executing Python code in the browser. Hosted by Google, it eliminates local software installation, providing access to computational resources (including GPUs and TPUs) for AI and data-intensive tasks. Integrated with Google Drive, Colab supports real-time collaboration, versioning, and sharing, with compatibility for libraries like TensorFlow, PyTorch, and Pandas, making it ideal for AI and data science workflows.
- **Anaconda**³: Cross-platform data science platform for Python and R, featuring package and environment management via Conda. Includes pre-installed libraries (e.g., NumPy, Pandas) and integrates with Jupyter and Spyder, streamlining workflows and reducing setup complexity for scalable projects.
- **Jupyter Notebook**⁴: Open-source, browser-based tool for creating interactive documents combining live code (e.g., Python), visualizations, equations, and text. Its cell-based structure supports iterative development, data exploration, and prototyping, with rich outputs (e.g., plots via Matplotlib, tables via Pandas) and export options (e.g., HTML, PDF). Hosted locally or via platforms like Colab, it enhances collaboration and reproducibility in data science and research.

¹<https://www.python.org/>

²<https://colab.research.google.com/>

³<https://www.anaconda.com/>

⁴<https://jupyter.org/>

Bibliography

- [1] A. Phinyomark, P. Phukpattaranont, and C. Limsakul, "Feature Extraction and Selection for EMG Signal Classification," *Expert Syst. Appl.*, vol. 39, no. 5, pp. 7420–7431, Apr. 2012, doi: 10.1016/j.eswa.2012.01.102.
- [2] C. Castellini, P. Artemiadis, M. Wininger, A. Ajoudani, M. Alimusaj, A. Bichi, B. Caputo, W. Craelius, S. Dosen, K. Englehart, D. Farina, A. Gijsberts, S. B. Godfrey, L. J. Hargrove, M. Ison, T. Kuiken, M. Markovic, P. M. Pilarski, R. Rupp, and E. Scheme, "Effect of EMG Signal Processing on the Estimation of Muscle Force," *J. Neural Eng.*, vol. 9, no. 4, Aug. 2012, doi: 10.1088/1741-2560/9/4/046005.
- [3] E. Scheme and K. Englehart, "Electromyogram Pattern Recognition for Control of Powered Upper-Limb Prostheses," *J. Rehabil. Res. Dev.*, vol. 48, no. 6, pp. 643–660, 2011, doi: 10.1682/JRRD.2010.09.0177.
- [4] M. Atzori, A. Gijsberts, C. Castellini, B. Caputo, A. G. Mittaz Hager, S. Elsig, G. Giatsidis, F. Bassetto, and H. Müller, "Electromyography data for non-invasive naturally-controlled robotic hand prostheses," *Sci. Data*, vol. 1, Art. no. 140053, Dec. 2014, doi: 10.1038/sdata.2014.53.
- [5] F. Negro, S. Muceli, A. M. Castronovo, A. Holobar, and D. Farina, "Multi-channel EMG processing for motor unit firing pattern analysis," *J. Neurophysiol.*, vol. 116, no. 5, pp. 2265–2273, Nov. 2016, doi: 10.1152/jn.00282.2016.
- [6] S. Pizzolato, L. Tagliapietra, M. Cognolato, M. Reggiani, H. Müller, and M. Atzori, "Comparison of six electromyography acquisition setups on hand movement classification tasks," *PLoS ONE*, vol. 12, no. 10, Oct. 2017, doi: 10.1371/journal.pone.0186132.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 5998–6008, 2017.
- [8] Z. Huang, Y. Xu, H. Zhang, X. Zhang, L. Bai, and K. Zhao, "TSFEDL: A Python Library for Time Series Spatio-Temporal Feature Extraction and Prediction Using Deep Learning," *Neurocomputing*, vol. 518, pp. 223–224, Jan. 2023, doi: 10.1016/j.neucom.2022.11.037.

- [9] Y. Zhao, L. Jiang, Y. Zhang, J. Yang, and Z. Zhang, "Adaptive Graph Transformer Network for Skeletal EMG-Based Motion Intent Recognition," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 31, pp. 2601–2611, 2023, doi: 10.1109/TNSRE.2023.3273849.
- [10] H. G. Lee, J. H. Chang, and S. W. Lee, "Subject-Independent Deep Architecture for sEMG-Based Recognition," *Sensors*, vol. 22, no. 3, p. 1160, Feb. 2022, doi: 10.3390/s22031160.
- [11] O. Javed, K. A. Maldonado, and R. Ashmyan, "Anatomy, shoulder and upper limb, muscles," StatPearls Publishing, Jul. 24, 2023. [Online]. Available: <https://www.ncbi.nlm.nih.gov/books/NBK537236/>. [Accessed: Feb. 18, 2025].
- [12] *The Role of Protein and Amino Acids in Sustaining and Enhancing Performance*. Committee on Military Nutrition Research, Food and Nutrition Board, Institute of Medicine. National Academy Press, 1999.
- [13] *Neural Contributions to Muscle Fatigue: From the Brain to the Muscle and Back Again*. Janet L. Taylor; Markus Amann; Jacques Duchateau; Romain Meeusen; Charles L. Rice. Medicine & Science in Sports & Exercise, 2016.
- [14] Brigham and Women's Hospital, "EMG: Detecting Neuromuscular Abnormalities," Brigham and Women's Hospital. [Online]. Available: <https://www.brighamandwomens.org/neurology/neuromuscular-diseases/electromyography>. [Accessed: Jan. 25, 2025].
- [15] T. Haab, P. Leinen, and P. Burkey, 'Role and effectiveness of surface EMG feedback in sports and orthopedic rehabilitation: a systematic review', *Exploration of Musculoskeletal Diseases*, vol. 2, no. 5, pp. 391–407, 2024.
- [16] J. Tchimino, R. L. Hansen, P. H. Jørgensen, J. Dideriksen, and S. Dosen, "Application of EMG feedback for hand prosthesis control in high-level amputation: a case study," *Sci Rep*, vol. 14, no. 1, p. 31676, Dec. 2024. doi: 10.1038/s41598-024-80828-x.
- [17] *Advanced Bioelectrical Signal Processing Methods: Past, Present, and Future Approach—Part III: Other Biosignals*. Radek Martinek; Martina Ladrova; Michaela Sidikova; Rene Jaros; Khosrow Behbehani; Radana Kahankova; Aleksandra Kawala-Sterniuk. Published: 10 September 2021.
- [18] *A novel method for automated EMG decomposition and MUAP classification*. C.D. Katsis; Y. Goletsis; A. Likas; D.I. Fotiadis; I. Sarvas. Available online 27 December 2005.
- [19] *Important Factors in Surface EMG Measurement*. Dr. Scott Day. Bortec Biomedical Incorporated, n.d. [Additional bibliographic details needed].
- [20] *MYOBOCK® Electrodes*. Otto Bock HealthCare. (n.d.).

- [21] *Delsys – Wearable Sensors for Movement Sciences*. Delsys. Retrieved on January 2, 2025 from <https://delsys.com/>.
- [22] *The ABC of EMG: A Practical Introduction to Kinesiological Electromyography*. Konrad, Peter. (2005).
- [23] *Signal Acquisition Using Surface EMG and Circuit Design Considerations for Robotic Prosthesis*. M. Zahak. In: *Computational Intelligence in Electromyography Analysis – A Perspective on Current Applications and Future Challenges*. InTech, October 17, 2012. doi: 10.5772/52556.
- [24] *Needle Electromyography: Basic Concepts*. Rubin, D. I. (2019). Handbook of Clinical Neurology, 160, 243–256. DOI: <https://doi.org/10.1016/B978-0-444-64032-1.00016-3>. PMID: 31277852.
- [25] G. Kamen and D. A. Gabriel, *Essentials of Electromyography*. Human Kinetics, 2010.
- [26] R. H. Chowdhury, M. B. Reaz, M. A. Ali, A. A. Bakar, K. Chellappan, and T. G. Chang, "Surface electromyography signal processing and classification techniques," **Sensors (Basel)**, vol. 13, no. 9, pp. 12431-12466, Sep. 2013, doi: 10.3390/s130912431.
- [27] M. Ortiz-Catalan, R. Brånemark, and B. Håkansson, 'BioPatRec: A modular research platform for the control of artificial limbs based on pattern recognition algorithms', *Source code for biology and medicine*, vol. 8, p. 11, 04 2013.
- [28] Y. Du, W. Jin, W. Wei, Y. Hu, and W. Geng, 'Surface EMG-Based Inter-Session Gesture Recognition Enhanced by Deep Domain Adaptation', *Sensors*, vol. 17, no. 3, 2017.
- [29] N. Krilova, I. Kastalskiy, V. Kazantsev, V. Makarov, and S. Lobov. "EMG Data for Gestures," UCI Machine Learning Repository, 2018. [Online]. Available: <https://doi.org/10.24432/C5ZP5C>.
- [30] *Attention Is All You Need*. Ashish Vaswani; Noam Shazeer; Niki Parmar; Jakob Uszkoreit; Llion Jones; Aidan N. Gomez; Lukasz Kaiser; Illia Polosukhin. Published: June 12, 2017.
- [31] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', *arXiv [cs.CL]*. 2019.
- [32] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun, *Transformers in Time Series: A Survey*,
- [33] Xingguang Yan, Liqiang Lin, Niloy J. Mitra, Dani Lischinski, Daniel Cohen-Or, and Hui Huang, *ShapeFormer: Transformer-based Shape Completion via Sparse Representation*, *arXiv:2201.10326 [cs.CV]*, 2022.

- [34] Zhou, H., Zhang, S., Peng, J., *et al.* “Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(12), pp. 11106–11115, 2021.
- [35] M. Montazerin, E. Rahimian, F. Naderkhani, S. F. Atashzar, and A. Mohammadi, “Transformer-based hand gesture recognition via high-density EMG signals: From instantaneous recognition to fusion of motor unit spike trains,” *arXiv preprint arXiv:2212.00743*, Nov. 2022, doi:10.48550/arXiv.2212.00743.
- [36] N. K. Karnam, S. R. Dubey, A. C. Turlapaty, and B. Gokaraju, “EMGHand-Net: A Hybrid CNN and Bi-LSTM Architecture for Hand Activity Classification using Surface EMG Signals,” *Biocybernetics and Biomedical Engineering*, vol. 42, no. 1, pp. 325–340, Feb. 2022, doi:10.1016/j.bbe.2022.02.005.
- [37] W. Zhang, T. Zhao, J. Zhang, and Y. Wang, “LST-EMG-Net: Long short-term transformer feature fusion network for sEMG gesture recognition,” *Frontiers in Neurorobotics*, vol. 17, art. 1127338, Feb. 28, 2023, doi:10.3389/fnbot.2023.1127338.
- [38] E. Rahimian, S. Zabihi, A. Asif, D. Farina, S. F. Atashzar, and A. Mohammadi, “Hand gesture recognition using temporal convolutions and attention mechanism,” *arXiv preprint arXiv:2110.08717*, Oct. 17, 2021, doi:10.48550/arXiv.2110.08717.
- [39] Xie, Baao (2021). Surface EMG based hand gesture recognition using hybrid deep learning networks. Loughborough University. Thesis. <https://doi.org/10.26174/thesis.lboro.16866844.v1>
- [40] U. Côté-Allard, C. L. Fall, A. Drouin, A. Campeau-Lecours, C. Gosselin, K. Glette, F. Laviolette, and B. Gosselin, “Deep learning for electromyographic hand gesture signal classification using transfer learning,” *arXiv preprint arXiv:1801.07756*, Jan. 26, 2019, doi:10.48550/arXiv.1801.07756.
- [41] M. Atzori, A. Gijsberts, C. Castellini, B. Caputo, A. G. Mittaz-Hager, S. Elsig, G. Giatsidis, F. Bassetto, and H. Müller, “The Ninapro database: a resource for sEMG naturally controlled robotic hand prosthetics,” **Scientific Data**, vol. 1, p. 140053, 2014. [Online]. Available: <https://ninapro.hevs.ch/>. [Accessed: Apr. 10, 2025].
- [42] J. Chang, A. Phinyomark, and E. Scheme, ‘Assessment of EMG Benchmark Data for Gesture Recognition Using the NinaPro Database’, in 2020 42nd Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC), 2020, pp. 3339–3342.
- [43] Electrical4U, ”Band Pass Filter: What is it? (Circuit, Design & Applications),” Electrical4U. [Online]. Available: <https://www.electrical4u.com/band-pass-filter/> [Accessed: apr. 26, 2025].

- [44] L. McManus, G. De Vito, and M. M. Lowery, ‘Analysis and biophysics of surface EMG for physiotherapists and kinesiologists: Toward a common language with rehabilitation engineers’, *Front. Neurol.*, vol. 11, p. 576729, Oct. 2020.
- [45] V. Lanzani, C. Brambilla, and A. Scano, ‘A methodological scoping review on EMG processing and synergy-based results in muscle synergy studies in Parkinson’s disease’, *Front. Bioeng. Biotechnol.*, vol. 12, p. 1445447, 2024.
- [46] Electrical4U, “Band Stop Filter and Notch Filter: Design and Applications,” Electrical4U. [Online]. Available: <https://www.electrical4u.com/band-stop-notch-filter/> [Accessed: apr.26, 2025].
- [47] M. Boyer, L. Bouyer, J.-S. Roy, and A. Campeau-Lecours, ‘Reducing noise, artifacts and interference in single-channel EMG signals: A review’, *Sensors (Basel)*, vol. 23, no. 6, p. 2927, Mar. 2023.
- [48] M. Atzori et al., ‘Building the NINAPRO Database: A Resource for the Biorobotics Community’, 12 2012.
- [49] Dharmaraj, *Convolutional Neural Networks (CNN) Architectures Explained*, Medium, [Online]. Available: <https://rb.gy/1ja3zy>.
- [50] Tomasz Janaszka, Mariusz Budziński, *Convolutional Networks for Time Series Classification*, CodiLime Blog, [Online]. Available: <https://codilime.com/blog/convolutional-networks-for-time-series-classification/>.
- [51] Shadman Sakib^{*}, Nazib Ahmed[#], Ahmed Jawad Kabir[@], and Hridon Ahmed^{\$}, *An Overview of Convolutional Neural Network: Its Architecture and Applications*
- [52] Sirine Amrane, *How do you beat overfitting in machine learning?*, Medium, [Online]. Available: <https://sirineamrane.medium.com/how-do-you-beat-overfitting-in-machine-learning-cf839265b345>.
- [53] Sirine Amrane, *Activation Functions in Classification: Part 1 - Softmax and Sigmoid*, Medium, [Online]. Available: <https://rb.gy/rqrx2p>.
- [54] Sirine Amrane, *Recurrent Neural Network (RNN) Architecture Explained*, Medium, [Online]. Available: <https://medium.com/@poudelsushmita878/recurrent-neural-network-rnn-architecture-explained-1d69560541ef>.
- [55] *Take A Shortcut Back: Mitigating the Gradient Vanishing for Training Spiking Neural Networks.* Yufei Guo; Yuanpei Chen; Zecheng Hao; Weihang Peng; Zhou Jie; Yuhang Zhang; Xiaode Liu; Zhe Ma. *Advances in Neural Information Processing Systems*, 2024.
- [56] *Deep Learning for Time Series Classification: A Review.* H. Ismail Fawaz; Germain Forestier; Jonathan Weber; Lhassane Idoumghar; Pierre-Alain Muller. *Data Mining and Knowledge Discovery*, 2019. doi: 10.1007/s10618-019-00619-1.

- [57] *InceptionTime: Finding AlexNet for Time Series Classification*. H. Ismail Fawaz; Benjamin Lucas; Germain Forestier; Charlotte Pelletier; Daniel F. Schmidt; Jonathan Weber; Geoffrey I. Webb; Lhassane Idoumghar; Pierre-Alain Muller; François Petitjean. Data Mining and Knowledge Discovery, 2020. doi: 10.1007/s10618-020-00710-y.
- [58] *Deep Convolutional Neural Networks on Multichannel Time Series for Human Activity Recognition*. Jian Bo Yang; Minh Nhut Nguyen; Phyo Phyo San; Xiao Li Li; Shonali Krishnaswamy. In Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence (IJCAI 2015), 2015.
- [59] *Deep Learning for Time Series Classification: A Review*. H. Ismail Fawaz; Germain Forestier; Jonathan Weber; Lhassane Idoumghar; Pierre-Alain Muller. Data Mining and Knowledge Discovery, 2019. doi: 10.1007/s10618-019-00619-1.
- [60] *InceptionTime: Finding AlexNet for Time Series Classification*. H. Ismail Fawaz; Benjamin Lucas; Germain Forestier; Charlotte Pelletier; Daniel F. Schmidt; Jonathan Weber; Geoffrey I. Webb; Lhassane Idoumghar; Pierre-Alain Muller; François Petitjean. Data Mining and Knowledge Discovery, 2020. doi: 10.1007/s10618-020-00710-y.
- [61] *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting*. Haoyi Zhou; Shanghang Zhang; Jieqi Peng; Shuai Zhang; Jianxin Li; Hui Xiong; Wancai Zhang. arXiv preprint arXiv:2012.07436, 2021.